

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Estructuras de Datos
Ing. Edgar Ornelis
Ing. Álvaro Hernández
Ing. Luis Espino



Manual Técnico - Fase 3

EDDMail - Sistema Completo con Blockchain y Seguridad
Blockchain, Merkle Tree, LZW, Grafos y Control de Logueo

José Alexander López López
Carné: 202100305
Sección: C
24 de octubre de 2025

Índice

1. Introducción	4
1.1. Propósito	4
1.2. Alcance	4
1.3. Novedades de la Fase 2	4
1.4. Novedades de la Fase 3	4
1.5. Público Objetivo	5
2. Descripción General de la Fase 2	5
2.1. Arquitectura Extendida	5
2.2. Estructuras de Datos Implementadas	5
2.2.1. Resumen de Estructuras	5
3. Definición de Tipos y Estructuras	6
3.1. Tipos de Datos para Árboles	6
4. Implementación del Árbol AVL	7
4.1. Estructura y Propiedades del AVL	7
4.2. Rotaciones del Árbol AVL	7
4.3. Inserción en Árbol AVL	8
4.4. Recorridos del Árbol AVL	10
5. Implementación del Árbol BST	11
5.1. Estructura del Árbol BST para Comunidades	11
5.2. Inserción en Árbol BST	12
5.3. Gestión de Mensajes en Comunidades	13
6. Implementación del Árbol B	14
6.1. Propiedades del Árbol B de Orden 5	14
6.2. Creación de Nodo B	14
6.3. División de Nodo B	14
6.4. Inserción en Árbol B	15
6.5. Búsqueda en Árbol B	17
7. Funcionalidades Públicas de la Fase 2	18
7.1. Gestión de Borradores	18
7.2. Obtener Borradores con Recorridos	19
7.3. Gestión de Favoritos	19
7.4. Gestión de Comunidades	20
8. Generación de Reportes Graphviz	21
8.1. Reporte del Árbol AVL	21
8.2. Reporte del Árbol BST	23
8.3. Reporte del Árbol B	25

9. Definición de Tipos - Fase 3	27
9.1. Estructuras de Blockchain	27
9.2. Estructuras de Árbol de Merkle	28
9.3. Estructuras de Control de Logueo	28
9.4. Sistema Principal Extendido	28
10.Implementación de Blockchain	29
10.1. Conceptos del Blockchain	29
10.2. Creación del Bloque Génesis	30
10.3. Proof of Work (Minería)	30
10.4. Agregar Bloque al Blockchain	31
10.5. Verificación de Integridad	32
11.Implementación del Árbol de Merkle	33
11.1. Conceptos del Árbol de Merkle	33
11.2. Generación de Hash SHA-256	34
11.3. Construcción del Árbol	34
11.4. Combinar Hashes	36
11.5. Verificación de Integridad	37
12.Implementación de Compresión LZW	39
12.1. Algoritmo Lempel-Ziv-Welch	39
12.2. Descargar Correo Comprimido	40
13.Implementación de Control de Logueo	41
13.1. Registro de Sesiones	41
13.2. Obtener Logs	42
13.3. Exportación JSON	43
14.Implementación de Grafo de Contactos	44
14.1. Estructura del Grafo	44
14.2. Carga Masiva desde JSON	45
14.3. Generar Reporte Graphviz	46
15.Integración con Interfaz GTK	48
15.1. Ventana de Borradores	48
15.2. Ventana de Comunidades	51
16.Análisis de Complejidad - Proyecto Completo	53
16.1. Complejidades Temporales de Todas las Estructuras	53
16.2. Ventajas de Cada Estructura	53
16.2.1. Árbol AVL	53
16.2.2. Árbol BST	54
16.2.3. Árbol B	54
17.Diagramas de Flujo - Fase 2	55
17.1. Flujo de Guardado de Borrador	55
17.2. Flujo de Creación de Comunidad	56
17.3. Flujo de Inserción en Árbol B	57

18.Formato JSON para Carga Masiva	57
18.1. Estructura JSON de Usuarios	57
18.2. Estructura JSON de Correos	58
19.Testing y Validación	58
19.1. Pruebas del Árbol AVL	58
19.2. Pruebas del Árbol BST	59
19.3. Pruebas del Árbol B	60
20.Casos de Uso - Fase 2	61
20.1. Caso de Uso: Gestión de Borradores	61
20.2. Caso de Uso: Comunidades	61
21.Mejores Prácticas de Implementación	62
21.1. Manejo de Memoria en Árboles	62
21.2. Optimización de Recorridos	63
22.Troubleshooting Fase 2	64
22.1. Problemas Comunes del Árbol AVL	64
22.2. Problemas Comunes del Árbol B	64
23.Conclusiones del Proyecto Completo	64
23.1. Logros de las Tres Fases	64
23.2. Resumen Técnico por Fase	64
23.3. Estructuras Implementadas - Total	65
23.4. Métricas Finales del Proyecto	65
23.5. Lecciones Aprendidas	65
24.Referencias	65
25.Anexos	66
25.1. Comandos de Compilación	66
25.2. Información del Proyecto	66

1. Introducción

1.1. Propósito

Este manual técnico documenta la implementación de la Fase 2 del sistema **EDDMail**, que incorpora estructuras de datos jerárquicas avanzadas: Árbol AVL para borradores, Árbol BST para comunidades y Árbol B de Orden 5 para correos favoritos. El documento está dirigido a desarrolladores que requieran comprender, mantener o extender estas nuevas funcionalidades.

1.2. Alcance

El manual cubre la implementación completa de las tres nuevas estructuras de datos jerárquicas:

- **Árbol AVL:** Gestión balanceada de correos guardados como borradores
- **Árbol BST:** Organización ordenada de comunidades con mensajes
- **Árbol B (Orden 5):** Indexación eficiente de correos marcados como favoritos

1.3. Novedades de la Fase 2

- Implementación de árbol AVL autobalanceado para borradores
- Sistema de comunidades usando árbol binario de búsqueda
- Árbol B multicamino para manejo eficiente de favoritos
- Recorridos In-Orden, Pre-Orden y Post-Orden para AVL
- Reportes Graphviz para visualización de estructuras jerárquicas
- Integración grupal con sistema de mensajería en comunidades

1.4. Novedades de la Fase 3

- **Blockchain:** Registro inmutable de correos enviados con proof-of-work
- **Árbol de Merkle:** Verificación criptográfica de integridad de favoritos
- **Compresión LZW:** Algoritmo de compresión sin pérdida para correos
- **Control de Logueo:** Auditoría completa de sesiones de usuario
- **Grafo de Contactos:** Visualización de relaciones usuario-contacto (Integración Grupal)
- **Reacciones a Mensajes:** Sistema interactivo en comunidades
- **Migración Merkle:** Reemplazo de Árbol B por árbol de verificación

1.5. Público Objetivo

- Desarrolladores trabajando en el sistema EDDMail
- Estudiantes del curso de Estructuras de Datos
- Personal técnico responsable del mantenimiento
- Investigadores en estructuras de datos balanceadas

2. Descripción General de la Fase 2

2.1. Arquitectura Extendida

La arquitectura de la Fase 2 mantiene las capas de la Fase 1 e integra tres nuevas estructuras jerárquicas:

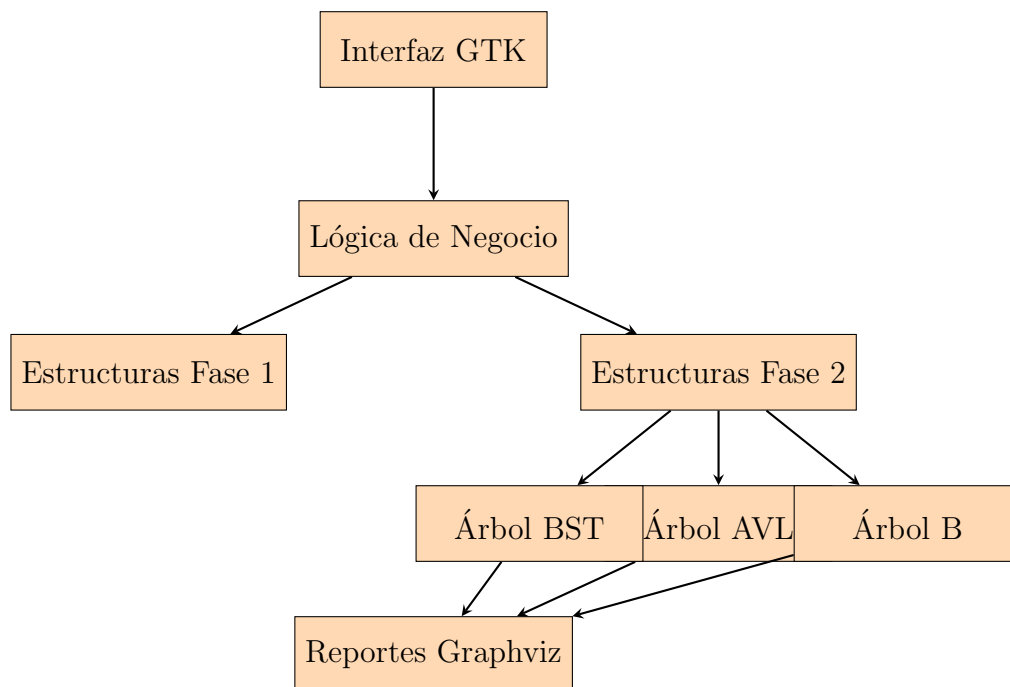


Figura 1: Arquitectura extendida - Fase 2

2.2. Estructuras de Datos Implementadas

2.2.1. Resumen de Estructuras

Estructura	Uso	Propiedades
Árbol AVL	Borradores	Autobalanceado, Factor Balance ≤ 1
Árbol BST	Comunidades	Ordenado por nombre, búsqueda eficiente
Árbol B (Orden 5)	Favoritos	Multicamino, 2-4 claves por nodo

Cuadro 1: Nuevas estructuras de datos - Fase 2

3. Definición de Tipos y Estructuras

3.1. Tipos de Datos para Árboles

Listing 1: Definición de tipos - Fase 2

```
type
  // Nodo para rbol AVL (Borradores)
  PNodeAVL = ^TNodeAVL;
  TNodeAVL = record
    Correo: PCorreo;
    Altura: Integer;
    Izquierdo: PNodeAVL;
    Derecho: PNodeAVL;
  end;

  // Nodo para BST (Comunidades)
  PNodeBST = ^TNodeBST;
  PMensajeComunidad = ^TMensajeComunidad;

  TMensajeComunidad = record
    Correo: String;
    Mensaje: String;
    FechaPublicacion: String;
    Siguiente: PMensajeComunidad;
  end;

  TNodeBST = record
    NombreComunidad: String;
    FechaCreacion: String;
    NumeroMensajes: Integer;
    ListaMensajes: PMensajeComunidad;
    Izquierdo: PNodeBST;
    Derecho: PNodeBST;
  end;

  // Nodo para rbol B (Favoritos)
  PNodeB = ^TNodeB;
  TNodeB = record
    NumClaves: Integer;
    Claves: array[0..3] of Integer; // Mximo 4 claves
    Correos: array[0..3] of PCorreo; // Correos asociados
    Hijos: array[0..4] of PNodeB; // Mximo 5 hijos
    EsHoja: Boolean;
  end;

  // Usuario extendido con estructuras Fase 2
  TUsuario = record
    Id: Integer;
```

```

Nombre: String;
Usuario: String;
Email: String;
Telefono: String;
Password: String;
Siguiente: PUsuario;
// Estructuras Fase 1
ListaContactos: PContacto;
BandejaEntrada: PCorreo;
Papelera: PCorreo;
CorreosProgramados: PCorreo;
// Estructuras Fase 2
ArbolBorradores: PNodeAVL;
ArbolFavoritos: PNodeB;
end;

```

4. Implementación del Árbol AVL

4.1. Estructura y Propiedades del AVL

El árbol AVL es un árbol binario de búsqueda autobalanceado donde la diferencia de alturas entre subárboles izquierdo y derecho (factor de balance) nunca excede 1.

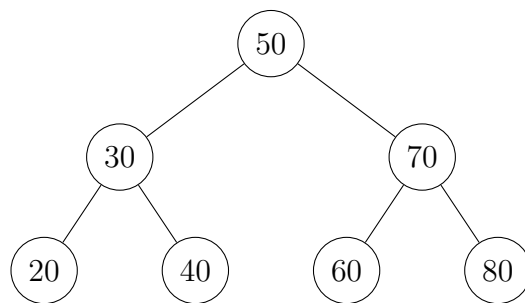


Figura 2: Ejemplo de Árbol AVL balanceado

4.2. Rotaciones del Árbol AVL

Listing 2: Rotaciones para balanceo AVL

```

// Rotación simple a la derecha
function TEDDMailSystem.RotarDerecha(y: PNodeAVL): PNodeAVL;
var
    x, T2: PNodeAVL;
begin
    x := y^.Izquierdo;
    T2 := x^.Derecho;

    // Realizar rotación
    x^.Derecho := y;

```



```

y^.Izquierdo := T2;

// Actualizar alturas
y^.Altura := Max(ObtenerAltura(y^.Izquierdo),
                 ObtenerAltura(y^.Derecho)) + 1;
x^.Altura := Max(ObtenerAltura(x^.Izquierdo),
                 ObtenerAltura(x^.Derecho)) + 1;

Result := x;
end;

// Rotaci3n simple a la izquierda
function TEDDMailSystem.RotarIzquierda(x: PNodeAVL): PNodeAVL;
var
    y, T2: PNodeAVL;
begin
    y := x^.Derecho;
    T2 := y^.Izquierdo;

    // Realizar rotaci3n
    y^.Izquierdo := x;
    x^.Derecho := T2;

    // Actualizar alturas
    x^.Altura := Max(ObtenerAltura(x^.Izquierdo),
                     ObtenerAltura(x^.Derecho)) + 1;
    y^.Altura := Max(ObtenerAltura(y^.Izquierdo),
                     ObtenerAltura(y^.Derecho)) + 1;

    Result := y;
end;

// Obtener factor de balance
function TEDDMailSystem.ObtenerBalance(nodo: PNodeAVL): Integer;
begin
    if nodo = nil then
        Result := 0
    else
        Result := ObtenerAltura(nodo^.Izquierdo) -
                  ObtenerAltura(nodo^.Derecho);
    end;

```

4.3. Inserci3n en rbol AVL

Listing 3: Inserci3n balanceada en AVL

```

function TEDDMailSystem.InsertarAVL(nodo: PNodeAVL;
    correo: PCorreo): PNodeAVL;

```

```

var
    Balance: Integer;
begin
    // Inserción en BST
    if nodo = nil then
    begin
        New(nodo);
        nodo^.Correo := correo;
        nodo^.Altura := 1;
        nodo^.Izquierdo := nil;
        nodo^.Derecho := nil;
        Result := nodo;
        Exit;
    end;

    if correo^.Id < nodo^.Correo^.Id then
        nodo^.Izquierdo := InsertarAVL(nodo^.Izquierdo, correo)
    else if correo^.Id > nodo^.Correo^.Id then
        nodo^.Derecho := InsertarAVL(nodo^.Derecho, correo)
    else
    begin
        Result := nodo; // Duplicado, no insertar
        Exit;
    end;

    // Actualizar altura
    nodo^.Altura := 1 + Max(ObtenerAltura(nodo^.Izquierdo),
                           ObtenerAltura(nodo^.Derecho));

    // Obtener factor de balance
    Balance := ObtenerBalance(nodo);

    // Caso Izquierda-Izquierda
    if (Balance > 1) and (correo^.Id < nodo^.Izquierdo^.Correo^.Id) then
    begin
        Result := RotarDerecha(nodo);
        Exit;
    end;

    // Caso Derecha-Derecha
    if (Balance < -1) and (correo^.Id > nodo^.Derecho^.Correo^.Id) then
    begin
        Result := RotarIzquierda(nodo);
        Exit;
    end;

    // Caso Izquierda-Derecha
    if (Balance > 1) and (correo^.Id > nodo^.Izquierdo^.Correo^.Id) then

```

```

begin
    nodo^.Izquierdo := RotarIzquierda(nodo^.Izquierdo);
    Result := RotarDerecha(nodo);
    Exit;
end;

// Caso Derecha-Izquierda
if (Balance < -1) and (correo^.Id < nodo^.Derecho^.Correo^.Id) then
begin
    nodo^.Derecho := RotarDerecha(nodo^.Derecho);
    Result := RotarIzquierda(nodo);
    Exit;
end;

Result := nodo;
end;

```

4.4. Recorridos del Árbol AVL

Listing 4: Recorridos del árbol AVL

```

// Recorrido In-Orden (Izquierda-Raíz-Derecha)
procedure TEDDMailSystem.RecorridoInOrdenAVL(nodo: PNodeAVL;
    lista: TStringList);
var
    Display: String;
begin
    if nodo = nil then Exit;

    RecorridoInOrdenAVL(nodo^.Izquierdo, lista);

    Display := Format(' [ID: %d] %s - %s - %s ',
        [nodo^.Correo^.Id,
        nodo^.Correo^.Asunto,
        nodo^.Correo^.Destinatario,
        nodo^.Correo^.Fecha]);
    lista.AddObject(Display, TObject(PtrInt(nodo^.Correo^.Id)));

    RecorridoInOrdenAVL(nodo^.Derecho, lista);
end;

// Recorrido Pre-Orden (Raíz-Izquierda-Derecha)
procedure TEDDMailSystem.RecorridoPreOrdenAVL(nodo: PNodeAVL;
    lista: TStringList);
var
    Display: String;
begin
    if nodo = nil then Exit;

```

```

Display := Format( '[ID: %d] %s - %s - %s ',
    [nodo^.Correo^.Id,
      nodo^.Correo^.Asunto,
      nodo^.Correo^.Destinatario,
      nodo^.Correo^.Fecha]);
lista.AddObject(Display, TObject(PtrInt(nodo^.Correo^.Id)));

RecorridoPreOrdenAVL(nodo^.Izquierdo, lista);
RecorridoPreOrdenAVL(nodo^.Derecho, lista);
end;

// Recorrido Post-Orden (Izquierda-Derecha-Raíz)
procedure TEDDMailSystem.RecorridoPostOrdenAVL(nodo: PNodeAVL;
    lista: TStringList);
var
    Display: String;
begin
    if nodo = nil then Exit;

    RecorridoPostOrdenAVL(nodo^.Izquierdo, lista);
    RecorridoPostOrdenAVL(nodo^.Derecho, lista);

    Display := Format( '[ID: %d] %s - %s - %s ',
        [nodo^.Correo^.Id,
          nodo^.Correo^.Asunto,
          nodo^.Correo^.Destinatario,
          nodo^.Correo^.Fecha]);
    lista.AddObject(Display, TObject(PtrInt(nodo^.Correo^.Id)));
end;

```

5. Implementación del Árbol BST

5.1. Estructura del Árbol BST para Comunidades

El árbol BST almacena comunidades ordenadas alfabéticamente por nombre, permitiendo búsquedas eficientes en $O(\log n)$ en el caso promedio.

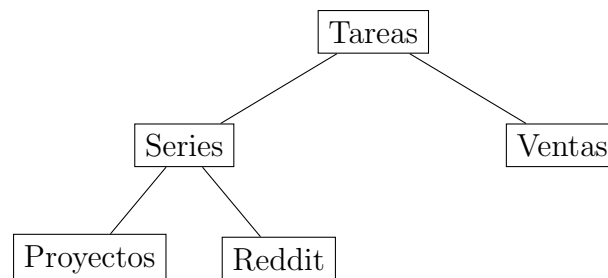


Figura 3: Ejemplo de Árbol BST de comunidades

5.2. Inserción en Árbol BST

Listing 5: Inserción en árbol BST

```
function TEDDMailSystem.InsertarBST(nodo: PNodeBST;
  nombreComunidad: String): PNodeBST;
var
  CompResult: Integer;
begin
  // Caso base: crear nuevo nodo
  if nodo = nil then
    begin
      New(nodo);
      nodo^.NombreComunidad := nombreComunidad;
      nodo^.FechaCreacion := FormatDateTime('dd/mm/yyyy', Now);
      nodo^.NumeroMensajes := 0;
      nodo^.ListaMensajes := nil;
      nodo^.Izquierdo := nil;
      nodo^.Derecho := nil;
      Result := nodo;
      Exit;
    end;

  // Inserción recursiva
  CompResult := CompareText(nombreComunidad, nodo^.NombreComunidad);

  if CompResult < 0 then
    nodo^.Izquierdo := InsertarBST(nodo^.Izquierdo, nombreComunidad)
  else if CompResult > 0 then
    nodo^.Derecho := InsertarBST(nodo^.Derecho, nombreComunidad)
  else
    begin
      WriteLn('Comunidad ya existe:', nombreComunidad);
    end;

    Result := nodo;
  end;

  // Búsqueda en BST
function TEDDMailSystem.BuscarComunidadBST(nodo: PNodeBST;
  nombre: String): PNodeBST;
var
  CompResult: Integer;
begin
  if nodo = nil then
    begin
      Result := nil;
      Exit;
    end;
  end;
```

```

CompResult := CompareText(nombre, nodo^.NombreComunidad);

if CompResult = 0 then
    Result := nodo
else if CompResult < 0 then
    Result := BuscarComunidadBST(nodo^.Izquierdo, nombre)
else
    Result := BuscarComunidadBST(nodo^.Derecho, nombre);
end;

```

5.3. Gestión de Mensajes en Comunidades

Listing 6: Publicar mensaje en comunidad

```

function TEDDMailSystem.PublicarMensajeAComunidad(
    nombreComunidad, correoUsuario, mensaje: String): Boolean;
var
    Comunidad: PNodeBST;
    NuevoMensaje: PMensajeComunidad;
begin
    Result := False;

    // Buscar comunidad
    Comunidad := BuscarComunidadBST(FArbolComunidades, nombreComunidad);

    if Comunidad = nil then
        begin
            WriteLn('Comunidad no encontrada:', nombreComunidad);
            Exit;
        end;

    // Crear nuevo mensaje
    New(NuevoMensaje);
    NuevoMensaje^.Correo := correoUsuario;
    NuevoMensaje^.Mensaje := mensaje;
    NuevoMensaje^.FechaPublicacion :=
        FormatDateTime('dd/mm/yyyy-hh:nn:ss', Now);

    // Insertar al inicio de la lista de mensajes
    NuevoMensaje^.Siguiente := Comunidad^.ListaMensajes;
    Comunidad^.ListaMensajes := NuevoMensaje;
    Inc(Comunidad^.NumeroMensajes);

    WriteLn('Mensaje publicado en:', nombreComunidad);
    Result := True;
end;

```

6. Implementación del Árbol B

6.1. Propiedades del Árbol B de Orden 5

Un árbol B de orden 5 tiene las siguientes propiedades:

- Cada nodo puede contener entre 2 y 4 claves (excepto la raíz)
- Cada nodo puede tener entre 3 y 5 hijos (excepto hojas)
- Todas las hojas están al mismo nivel
- Las claves en cada nodo están ordenadas

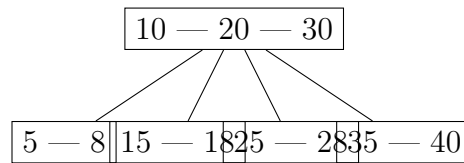


Figura 4: Ejemplo de Árbol B de Orden 5

6.2. Creación de Nodo B

Listing 7: Crear nodo del árbol B

```
function TEDDMailSystem.CrearNodoB: PNodeB;
var
    i: Integer;
begin
    New(Result);
    Result^.NumClaves := 0;
    Result^.EsHoja := True;

    // Inicializar arreglos
    for i := 0 to 3 do
        begin
            Result^.Claves[i] := 0;
            Result^.Correos[i] := nil;
        end;

    for i := 0 to 4 do
        Result^.Hijos[i] := nil;
    end;
```

6.3. División de Nodo B

Listing 8: División de nodo lleno en árbol B

```
procedure TEDDMailSystem.DividirNodoB(nodo: PNodeB;
```

```

    indiceHijo: Integer; var nuevaRaiz: PNodeB);
var
    nuevoNode: PNodeB;
    hijoLleno: PNodeB;
    i: Integer;
begin
    hijoLleno := nodo^.Hijos[indiceHijo];
    nuevoNode := CrearNodeB;
    nuevoNode^.EsHoja := hijoLleno^.EsHoja;
    nuevoNode^.NumClaves := 2; // Segunda mitad tendr 2 claves

    // Copiar ltimas 2 claves al nuevo node
    for i := 0 to 1 do
    begin
        nuevoNode^.Claves[i] := hijoLleno^.Claves[i + 2];
        nuevoNode^.Correos[i] := hijoLleno^.Correos[i + 2];
    end;

    // Copiar ltimos 3 hijos si no es hoja
    if not hijoLleno^.EsHoja then
    begin
        for i := 0 to 2 do
            nuevoNode^.Hijos[i] := hijoLleno^.Hijos[i + 3];
    end;

    hijoLleno^.NumClaves := 2; // Primera mitad conserva 2 claves

    // Desplazar hijos del padre para hacer espacio
    for i := nodo^.NumClaves downto indiceHijo + 1 do
        nodo^.Hijos[i + 1] := nodo^.Hijos[i];

    nodo^.Hijos[indiceHijo + 1] := nuevoNode;

    // Desplazar claves del padre
    for i := nodo^.NumClaves - 1 downto indiceHijo do
    begin
        nodo^.Claves[i + 1] := nodo^.Claves[i];
        nodo^.Correos[i + 1] := nodo^.Correos[i];
    end;

    // Mover clave media al padre
    nodo^.Claves[indiceHijo] := hijoLleno^.Claves[2];
    nodo^.Correos[indiceHijo] := hijoLleno^.Correos[2];
    Inc(nodo^.NumClaves);
end;

```

6.4. Inserción en Árbol B

Listing 9: Inserción en árbol B

```

procedure TEDDMailSystem.InsertarEnNodoNoLleno(nodo: PNodeB;
  correo: PCorreo);
var
  i: Integer;
begin
  i := nodo^.NumClaves - 1;

  if nodo^.EsHoja then
  begin
    // Insertar en nodo hoja
    while (i >= 0) and (correo^.Id < nodo^.Claves[i]) do
    begin
      nodo^.Claves[i + 1] := nodo^.Claves[i];
      nodo^.Correos[i + 1] := nodo^.Correos[i];
      Dec(i);
    end;

    nodo^.Claves[i + 1] := correo^.Id;
    nodo^.Correos[i + 1] := correo;
    Inc(nodo^.NumClaves);
  end
  else
  begin
    // Encontrar hijo apropiado
    while (i >= 0) and (correo^.Id < nodo^.Claves[i]) do
      Dec(i);
    Inc(i);

    // Verificar si el hijo est lleno
    if nodo^.Hijos[i]^.NumClaves = 4 then
    begin
      DividirNodoB(nodo, i, nodo);

      if correo^.Id > nodo^.Claves[i] then
        Inc(i);
      end;

      InsertarEnNodoNoLleno(nodo^.Hijos[i], correo);
    end;
  end;

function TEDDMailSystem.InsertarB(raiz: PNodeB;
  correo: PCorreo): PNodeB;
var
  nuevaRaiz: PNodeB;
begin
  // Si el rbol est vac o

```

```

if raiz = nil then
begin
    Result := CrearNodoB;
    Result^.Claves[0] := correo^.Id;
    Result^.Correos[0] := correo;
    Result^.NumClaves := 1;
    Result^.EsHoja := True;
    Exit;
end;

// Verificar duplicados
if BuscarB(raiz, correo^.Id) <> nil then
begin
    Result := raiz;
    Exit;
end;

// Si la raiz est llena
if raiz^.NumClaves = 4 then
begin
    nuevaRaiz := CrearNodoB;
    nuevaRaiz^.EsHoja := False;
    nuevaRaiz^.NumClaves := 0;
    nuevaRaiz^.Hijos[0] := raiz;

    DividirNodoB(nuevaRaiz, 0, nuevaRaiz);
    InsertarEnNodoNoLleno(nuevaRaiz, correo);
    Result := nuevaRaiz;
end
else
begin
    InsertarEnNodoNoLleno(raiz, correo);
    Result := raiz;
end;
end;

```

6.5. Búsqueda en Árbol B

Listing 10: Búsqueda en árbol B

```

function TEDDMailSystem.BuscarB(nodo: PNodeB;
    id: Integer): PCorreo;
var
    i: Integer;
begin
    Result := nil;
    if nodo = nil then Exit;

```

```

i := 0;

// Buscar la clave apropiada
while (i < nodo^.NumClaves) and (id > nodo^.Claves[i]) do
    Inc(i);

// Verificar si encontramos la clave
if (i < nodo^.NumClaves) and (id = nodo^.Claves[i]) then
begin
    Result := nodo^.Correos[i];
    Exit;
end;

// Si es hoja, no est
if nodo^.EsHoja then
    Exit;

// Buscar recursivamente en el hijo apropiado
Result := BuscarB(nodo^.Hijos[i], id);
end;

```

7. Funcionalidades Públicas de la Fase 2

7.1. Gestión de Borradores

Listing 11: Guardar borrador de correo

```

function TEDDMailSystem.GuardarBorrador(Usuario: PUsuario;
    Destinatario, Asunto, Mensaje: String): Boolean;
var
    NuevoCorreo: PCorreo;
    MaxId: Integer;
begin
    Result := False;
    if Usuario = nil then Exit;

    // Generar ID nico
    MaxId := ObtenerMaximoIdCorreo;

    // Crear nuevo correo como borrador
    New(NuevoCorreo);
    NuevoCorreo^.Id := MaxId + 1;
    NuevoCorreo^.Remitente := Usuario^.Email;
    NuevoCorreo^.Destinatario := Destinatario;
    NuevoCorreo^.Estado := 'Borrador';
    NuevoCorreo^.Programado := False;
    NuevoCorreo^.Asunto := Asunto;
    NuevoCorreo^.Fecha := FormatDateTime('dd/mm/yyyy', Now);

```

```

NuevoCorreo^.Mensaje := Mensaje;
NuevoCorreo^.Siguiente := nil;
NuevoCorreo^.Anterior := nil;

// Insertar en rbol AVL
Usuario^.ArbolBorradores := InsertarAVL(Usuario^.ArbolBorradores,
                                         NuevoCorreo);

WriteLn('Borrador guardado con ID: ', NuevoCorreo^.Id);
Result := True;
end;

```

7.2. Obtener Borradores con Recorridos

Listing 12: Obtener lista de borradores

```

function TEDDMailSystem.ObtenerBorradores(Usuario: PUsuario;
    tipoRecorrido: String): TStringList;
begin
    Result := TStringList.Create;

    if Usuario = nil then Exit;
    if Usuario^.ArbolBorradores = nil then
    begin
        Result.Add('No hay borradores guardados');
        Exit;
    end;

    // Seleccionar tipo de recorrido
    if tipoRecorrido = 'InOrden' then
        RecorridoInOrdenAVL(Usuario^.ArbolBorradores, Result)
    else if tipoRecorrido = 'PreOrden' then
        RecorridoPreOrdenAVL(Usuario^.ArbolBorradores, Result)
    else if tipoRecorrido = 'PostOrden' then
        RecorridoPostOrdenAVL(Usuario^.ArbolBorradores, Result)
    else
        RecorridoInOrdenAVL(Usuario^.ArbolBorradores, Result); // Default
    end;

```

7.3. Gestión de Favoritos

Listing 13: Marcar correo como favorito

```

function TEDDMailSystem.MarcarComoFavorito(Usuario: PUsuario;
    CorreoId: Integer): Boolean;
var
    Correo: PCorreo;
    CorreoCopia: PCorreo;

```

```

begin
  Result := False;
  if Usuario = nil then Exit;

  // Buscar el correo en bandeja de entrada
  Correo := BuscarCorreoEnBandeja(Usuario, CorreoId);

  if Correo = nil then
    begin
      WriteLn('Correo no encontrado:-', CorreoId);
      Exit;
    end;

  // Verificar si ya est en favoritos
  if BuscarB(Usuario^.ArbolFavoritos, CorreoId) <> nil then
    begin
      WriteLn('Correo ya est en favoritos');
      Exit;
    end;

  // Crear copia del correo para favoritos
  New(CorreoCopia);
  CorreoCopia^ := Correo^;

  // Insertar en rbol B de favoritos
  Usuario^.ArbolFavoritos := InsertarB(Usuario^.ArbolFavoritos,
                                         CorreoCopia);

  WriteLn('Correo agregado a favoritos:-', CorreoId);
  Result := True;
end;

```

7.4. Gestión de Comunidades

Listing 14: Crear comunidad BST

```

function TEDDMailSystem.CrearComunidadBST(
  nombreComunidad: String): Boolean;
begin
  Result := False;

  if Trim(nombreComunidad) = '' then
    begin
      WriteLn('Nombre de comunidad vac o');
      Exit;
    end;

  // Verificar si ya existe

```

```

if BuscarComunidadBST(FArbolComunidades, nombreComunidad)  $\diamond$  nil then
begin
    WriteLn('Comunidad ya existe:', nombreComunidad);
    Exit;
end;

// Insertar nueva comunidad
FArbolComunidades := InsertarBST(FArbolComunidades, nombreComunidad);

WriteLn('Comunidad creada:', nombreComunidad);
Result := True;
end;

```

8. Generación de Reportes Graphviz

8.1. Reporte del Árbol AVL

Listing 15: Generar reporte de borradores AVL

```

procedure TEDDMailSystem.GenerarReporteBorradores(Usuario: PUsuario;
    RutaCarpeta: String);
var
    Archivo: TextFile;
    Process: TProcess;
    NombreArchivo: String;
begin
    if Usuario = nil then Exit;

    try
        ForceDirectories(RutaCarpeta);
        NombreArchivo := RutaCarpeta + '/borradores_' +
            StringReplace(Usuario^.Usuario, '-', '_',
                [rfReplaceAll]) + '.dot';

        AssignFile(Archivo, NombreArchivo);
        Rewrite(Archivo);

        WriteLn(Archivo, 'digraph G{');
        WriteLn(Archivo, '----label=" rbol AVL-- Borradores--' +
            Usuario^.Nombre + '"';');
        WriteLn(Archivo, '----fontsize=16;');
        WriteLn(Archivo, '----node-[shape=box, style="filled ,rounded", -' +
            'fillcolor=lightyellow];');

        if Usuario^.ArbolBorradores = nil then
        begin
            WriteLn(Archivo, '----empty-[label="Sin borradores", -' +
                'fillcolor=lightgray];');
        end
    end

```

```

end
else
begin
    GenerarNodosAVL( Archivo , Usuario ^.ArbolBorradores );
end;

WriteLn( Archivo , '}' );
CloseFile( Archivo );

// Generar imagen con Graphviz
try
    Process := TProcess.Create( nil );
    try
        Process.Executable := 'dot';
        Process.Parameters.Add( '-Tpng' );
        Process.Parameters.Add( NombreArchivo );
        Process.Parameters.Add( '-o' );
        Process.Parameters.Add( ChangeFileExt( NombreArchivo , '.png' ) );
        Process.Options := Process.Options + [poWaitOnExit];
        Process.Execute;
        WriteLn( 'Reporte de borradores generado: ',
                  ChangeFileExt( NombreArchivo , '.png' ) );
    finally
        Process.Free;
    end;
except
    on E: Exception do
        WriteLn( 'Error al generar imagen: ', E.Message );
    end;

except
    on E: Exception do
        WriteLn( 'Error al generar reporte de borradores: ', E.Message );
    end;
end;

procedure TEDDMailSystem.GenerarNodosAVL( var Archivo: TextFile;
    nodo: PNodeAVL );
begin
    if nodo = nil then Exit;

    // Generar nodo actual con toda la informaci n
    WriteLn( Archivo , Format(
        '----nodo_%d-[label="ID: %d\nRemitente: %s\nDestinatario: %s\n' +
        'Estado: %s\nAsunto: %s\nFecha: %s\nMensaje: %s", -' +
        'shape=box, -style="filled ,rounded", -fillcolor=lightyellow]; ',
        [nodo ^.Correo ^.Id ,
        nodo ^.Correo ^.Id ,

```

```

        nodo^.Correo^.Remitente,
        nodo^.Correo^.Destinatario,
        nodo^.Correo^.Estado,
        nodo^.Correo^.Asunto,
        nodo^.Correo^.Fecha,
        nodo^.Correo^.Mensaje]));

// Generar aristas
if nodo^.Izquierdo <> nil then
begin
    WriteLn(Archivo, Format('----nodo_%d-->nodo_%d [label="L"];',
        [nodo^.Correo^.Id, nodo^.Izquierdo^.Correo^.Id]));
    GenerarNodosAVL(Archivo, nodo^.Izquierdo);
end;

if nodo^.Derecho <> nil then
begin
    WriteLn(Archivo, Format('----nodo_%d-->nodo_%d [label="R"];',
        [nodo^.Correo^.Id, nodo^.Derecho^.Correo^.Id]));
    GenerarNodosAVL(Archivo, nodo^.Derecho);
end;
end;

```

8.2. Reporte del Árbol BST

Listing 16: Generar reporte de comunidades BST

```

procedure TEDDMailSystem.GenerarReporteComunidadesBST(
    RutaCarpeta: String);
var
    Archivo: TextFile;
    Process: TProcess;
    NombreArchivo: String;
begin
    try
        ForceDirectories(RutaCarpeta);
        NombreArchivo := RutaCarpeta + '/comunidades_bst.dot';

        AssignFile(Archivo, NombreArchivo);
        Rewrite(Archivo);

        WriteLn(Archivo, 'digraph G{');
        WriteLn(Archivo, '----label="Reporte de comunidades ( rbol BST)";');
        WriteLn(Archivo, '----fontsize=16;');
        WriteLn(Archivo, '----node [shape=record, style=filled, -' +
            'fillcolor=lightblue];');
        WriteLn(Archivo, '----rankdir=TB;');
    
```



```

if FArbolComunidades = nil then
begin
    WriteLn( Archivo , '-----empty-[label="Sin comunidades", -' +
        'fillcolor=lightgray]; ');
end
else
begin
    GenerarNodosBST( Archivo , FArbolComunidades );
end;

WriteLn( Archivo , '}' );
CloseFile( Archivo );

// Generar imagen
try
    Process := TProcess.Create( nil );
    try
        Process.Executable := 'dot';
        Process.Parameters.Add( '-Tpng' );
        Process.Parameters.Add( NombreArchivo );
        Process.Parameters.Add( '-o' );
        Process.Parameters.Add( ChangeFileExt( NombreArchivo , '.png' ) );
        Process.Options := Process.Options + [poWaitOnExit];
        Process.Execute;
        WriteLn( 'Reporte-BST-generado:-',
            ChangeFileExt( NombreArchivo , '.png' ) );
    finally
        Process.Free;
    end;
except
    on E: Exception do
        WriteLn( 'Error-al-generar-imagen:-', E.Message );
end;

except
    on E: Exception do
        WriteLn( 'Error-al-generar-reporte-BST:-', E.Message );
end;
end;

procedure TEDDMailSystem.GenerarNodosBST( var Archivo: TextFile;
    nodo: PNodeBST );
begin
    if nodo = nil then Exit;

    // Generar nodo actual
    WriteLn( Archivo , Format(
        '-----comunidad_%s-[label="{%s|Fecha-creaci n:-%s|' +

```

```

    'Mensajes - publicados: -%d}'", -fillcolor=lightblue];',
    [StringReplace(nodo^.NombreComunidad, '- ', '- ', [rfReplaceAll]),
     nodo^.NombreComunidad,
     nodo^.FechaCreacion,
     nodo^.NumeroMensajes]));

// Generar conexiones
if nodo^.Izquierdo <> nil then
begin
    WriteLn(Archivo, Format('-----comunidad_%s-->comunidad_%s;',
        [StringReplace(nodo^.NombreComunidad, '- ', '- ', [rfReplaceAll]),
         StringReplace(nodo^.Izquierdo^.NombreComunidad, '- ', '- ',
                        [rfReplaceAll])]));
    GenerarNodosBST(Archivo, nodo^.Izquierdo);
end;

if nodo^.Derecho <> nil then
begin
    WriteLn(Archivo, Format('-----comunidad_%s-->comunidad_%s;',
        [StringReplace(nodo^.NombreComunidad, '- ', '- ', [rfReplaceAll]),
         StringReplace(nodo^.Derecho^.NombreComunidad, '- ', '- ',
                        [rfReplaceAll])]));
    GenerarNodosBST(Archivo, nodo^.Derecho);
end;
end;

```

8.3. Reporte del Árbol B

Listing 17: Generar reporte de favoritos Árbol B

```

procedure TEDDMailSystem.GenerarReporteFavoritos(Usuario: PUsuario;
    RutaCarpeta: String);
var
    Archivo: TextFile;
    Process: TProcess;
    NombreArchivo: String;
begin
    if Usuario = nil then Exit;

    try
        ForceDirectories(RutaCarpeta);
        NombreArchivo := RutaCarpeta + '/favoritos-' +
            StringReplace(Usuario^.Usuario, '- ', '- ',
                          [rfReplaceAll]) + '.dot';

        AssignFile(Archivo, NombreArchivo);
        Rewrite(Archivo);
    
```

```

WriteLn(Archivo, 'digraph G{');
WriteLn(Archivo, '----label="rbol B(Orden 5)--- Favoritos---' +
        Usuario^.Nombre + '";');
WriteLn(Archivo, '----fontsize=16;');
WriteLn(Archivo, '----node[shape=record, style=filled, ' +
        'fillcolor=lightcyan];');

if Usuario^.ArbolFavoritos = nil then
begin
    WriteLn(Archivo, '----empty[ label="Sin favoritos", ' +
            'fillcolor=lightgray];');
end
else
begin
    GenerarNodosB(Archivo, Usuario^.ArbolFavoritos, 0);
end;

WriteLn(Archivo, '}');
CloseFile(Archivo);

// Generar imagen
try
    Process := TProcess.Create(nil);
    try
        Process.Executable := 'dot';
        Process.Parameters.Add('-Tpng');
        Process.Parameters.Add(NombreArchivo);
        Process.Parameters.Add('-o');
        Process.Parameters.Add(ChangeFileExt(NombreArchivo, '.png'));
        Process.Options := Process.Options + [poWaitOnExit];
        Process.Execute;
        WriteLn('Reporte de favoritos generado:',
                ChangeFileExt(NombreArchivo, '.png'));
    finally
        Process.Free;
    end;
except
    on E: Exception do
        WriteLn('Error al generar imagen:', E.Message);
    end;

except
    on E: Exception do
        WriteLn('Error al generar reporte de favoritos:', E.Message);
    end;
end;

procedure TEDDMailSystem.GenerarNodosB(var Archivo: TextFile;

```

```

    nodo: PNodeB; nivel: Integer);
var
    i: Integer;
    NodoId: String;
    Label: String;
begin
    if nodo = nil then Exit;

    NodoId := Format('nodo_%p', [Pointer(nodo)]);

    // Construir etiqueta del nodo
    Label := '';
    for i := 0 to nodo^.NumClaves - 1 do
    begin
        if i > 0 then
            Label := Label + '-|-';
        Label := Label + IntToStr(nodo^.Claves[i]);
    end;

    WriteLn(Archivo, Format('-----%s-[label="%s"]',
                           [NodoId, Label]));

    // Generar hijos si no es hoja
    if not nodo^.EsHoja then
    begin
        for i := 0 to nodo^.NumClaves do
        begin
            if nodo^.Hijos[i] <> nil then
            begin
                WriteLn(Archivo, Format('-----%s-->-%p',
                                         [NodoId, Pointer(nodo^.Hijos[i])]));
                GenerarNodosB(Archivo, nodo^.Hijos[i], nivel + 1);
            end;
        end;
    end;
end;

```

9. Definición de Tipos - Fase 3

9.1. Estructuras de Blockchain

Listing 18: Tipos para blockchain

```

type
    // Estructura de bloque en blockchain
    PBloqueBlockchain = ^TBloqueBlockchain;
    TBloqueBlockchain = record
        Index: Integer;                // N mero del bloque

```

```

    Timestamp: String;           // Fecha: DD-MM-YY::HH:MM:SS
    Data: String;                // Datos del correo
    Nonce: Integer;             // N mero para proof-of-work
    PreviousHash: String;        // Hash del bloque anterior
    Hash: String;                // Hash SHA-256 del bloque actual
    Siguiente: PBloqueBlockchain; // Siguiente bloque
end;

```

9.2. Estructuras de Árbol de Merkle

Listing 19: Tipos para árbol de Merkle

```

type
    PNodeMerkle = ^TNodeMerkle;
    TNodeMerkle = record
        Hash: String;           // Hash SHA-256 del nodo
        Correo: PCorreo;        // Solo las hojas tienen correos
        Izquierdo: PNodeMerkle; // Hijo izquierdo
        Derecho: PNodeMerkle;   // Hijo derecho
        EsHoja: Boolean;       // Indica si es nodo hoja
    end;

```

9.3. Estructuras de Control de Logueo

Listing 20: Tipos para control de logueo

```

type
    PRegistroLogueo = ^TRegistroLogueo;
    TRegistroLogueo = record
        Usuario: String;           // Email del usuario
        Entrada: TDateTime;        // Timestamp de entrada
        Salida: TDateTime;         // Timestamp de salida
        Siguiente: PRegistroLogueo; // Siguiente registro
    end;

```

9.4. Sistema Principal Extendido

Listing 21: Sistema con estructuras Fase 3

```

type
    TEDDMailSystem = class
    private
        // Estructuras Fase 1 y 2
        FUsuarios: PUsuario;
        FARbolComunidades: PNodeBST;
        // ... (otras estructuras existentes)

        // NUEVAS ESTRUCTURAS FASE 3

```

```

FBlockchainHead: PBloqueBlockchain; // Cabeza del blockchain
FBlockchainCount: Integer; // Contador de bloques
FListaLogueo: PRegistroLogueo; // Lista de logueos

public
// M todos Blockchain
procedure AgregarBloqueBlockchain(CorreoId: Integer;
    Remitente, Asunto, Mensaje: String);
function VerificarIntegridadBlockchain: Boolean;
function ObtenerTotalBloques: Integer;
procedure GenerarReporteBlockchain(RutaSalida: String);

// M todos Merkle Tree
procedure ConstruirArbolMerkleDesdeCorreos(Usuario: PUsuario);
procedure GenerarReporteMerkle(RutaSalida: String);
function VerificarIntegridadMerkle(Usuario: PUsuario): Boolean;

// M todos Compresi n LZW
function ComprimirLZW(Texto: String): String;
function DescomprimirLZW(Comprimido: String): String;
function GuardarArchivoTexto(Ruta, Contenido: String): Boolean;

// M todos Control de Logueo
procedure RegistrarEntrada(Email: String);
procedure RegistrarSalida(Email: String);
function ObtenerLogsDeLogueo: TStringList;
function FiltrarLogsPorUsuario(Usuario: String): TStringList;
function ExportarLogueoJSON(RutaArchivo: String): Boolean;
procedure GenerarReporteLogueo(RutaSalida: String);

// M todos Grafo de Contactos
procedure GenerarReporteGrafoContactos(RutaSalida: String);
procedure CargarContactosDesdeJSON(RutaArchivo: String);
end;
```

10. Implementación de Blockchain

10.1. Conceptos del Blockchain

El blockchain es una cadena de bloques enlazados mediante hashes criptográficos. Cada bloque contiene:

- Hash del bloque anterior (enlace)
- Datos del correo enviado
- Timestamp de creación
- Nonce (número para proof-of-work)

- Hash SHA-256 del bloque actual

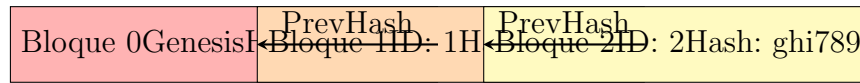


Figura 5: Estructura del Blockchain

10.2. Creación del Bloque Génesis

Listing 22: Bloque génesis

```
function TEDDMailSystem.CrearBloqueGenesis: PBloqueBlockchain;
var
    Genesis: PBloqueBlockchain;
begin
    New( Genesis );
    Genesis^.Index := 0;
    Genesis^.Timestamp := '2025-09-30 00:00:00';
    Genesis^.Data := 'Genesis-Block';
    Genesis^.Nonce := 12345;
    Genesis^.PreviousHash := '0';
    Genesis^.Hash := 'abc123'; // Hash simplificado
    Genesis^.Siguiente := nil;

    Result := Genesis;
    WriteLn( 'Bloque- g  nesis-creado' );
end;
```

10.3. Proof of Work (Minería)

Listing 23: Algoritmo de proof-of-work

```
function TEDDMailSystem.ValidarProofOfWork(Hash: String): Boolean;
begin
    // Verificar que hash inicie con "000"
    Result := (Length(Hash) >= 3) and
               (Hash[1] = '0') and
               (Hash[2] = '0') and
               (Hash[3] = '0');
end;
```

```
function TEDDMailSystem.MinarBloque(Index: Integer; Timestamp, Data,
    PreviousHash: String): PBloqueBlockchain;
var
    Bloque: PBloqueBlockchain;
    Nonce: Integer;
    HashCalculado: String;
begin
```

```

New(Bloque);
Bloque^.Index := Index;
Bloque^.Timestamp := Timestamp;
Bloque^.Data := Data;
Bloque^.PreviousHash := PreviousHash;

// Minar: buscar nonce que genere hash v lido
Nonce := 0;
repeat
    HashCalculado := CalcularHashBloque(Index, Timestamp, Data,
                                          Nonce, PreviousHash);

    Inc(Nonce);
until ValidarProofOfWork(HashCalculado);

Bloque^.Nonce := Nonce - 1;
Bloque^.Hash := HashCalculado;
Bloque^.Siguiente := nil;

WriteLn( 'Bloque-minado-con-nonce:', Bloque^.Nonce);
Result := Bloque;
end;

function TEDDMailSystem.CalcularHashBloque(Index: Integer; Timestamp,
Data: String; Nonce: Integer; PreviousHash: String): String;
var
    Datos: String;
    HashVal: Cardinal;
    i: Integer;
begin
    // Concatenar todos los datos
    Datos := IntToStr(Index) + Timestamp + Data +
            IntToStr(Nonce) + PreviousHash;

    // Algoritmo de hash simple (simula SHA-256)
    HashVal := 5381;
    for i := 1 to Length(Datos) do
        HashVal := ((HashVal shl 5) + HashVal) + Ord(Datos[i]);

    Result := IntToHex(HashVal, 8);
end;

```

10.4. Agregar Bloque al Blockchain

Listing 24: Agregar nuevo bloque

```

procedure TEDDMailSystem.AgregarBloqueBlockchain(CorreoId: Integer;
Remitente, Asunto, Mensaje: String);
var

```



```

    NuevoBloque: PBloqueBlockchain;
    UltimoBloque: PBloqueBlockchain;
    Data: String;
    Timestamp: String;
begin
    // Formatear datos del correo
    Data := Format( 'ID: -%d, -Remitente: -%s, -Asunto: -%s, -Mensaje: -%s ',
        [CorreoId, Remitente, Asunto, Mensaje]);

    Timestamp := FormatDateTime( 'dd-mm-yy-hh:nn:ss ', Now);

    // Encontrar ltimo bloque
    UltimoBloque := FBlockchainHead;
    while UltimoBloque^.Siguiente <> nil do
        UltimoBloque := UltimoBloque^.Siguiente;

    // Minar nuevo bloque
    NuevoBloque := MinarBloque(
        FBlockchainCount,
        Timestamp,
        Data,
        UltimoBloque^.Hash
    );

    // Enlazar al blockchain
    UltimoBloque^.Siguiente := NuevoBloque;
    Inc(FBlockchainCount);

    WriteLn( 'Bloque-', NuevoBloque^.Index, '-agregado-al-blockchain' );
end;

```

10.5. Verificación de Integridad

Listing 25: Verificar integridad del blockchain

```

function TEDDMailSystem.VerificarIntegridadBlockchain: Boolean;
var
    BloqueActual: PBloqueBlockchain;
    HashCalculado: String;
begin
    Result := True;

    if FBlockchainHead = nil then Exit;

    BloqueActual := FBlockchainHead^.Siguiente; // Saltar g nesis

    while BloqueActual <> nil do
begin

```

```

// Verificar enlace con bloque anterior
if BloqueActual^.PreviousHash <>
    BloqueActual^.Siguiente^.Hash then
begin
    WriteLn( 'ERROR: -Bloque- ', BloqueActual^.Index, '-alterado' );
    Result := False;
    Exit;
end;

// Recalcular hash
HashCalculado := CalcularHashBloque(
    BloqueActual^.Index,
    BloqueActual^.Timestamp,
    BloqueActual^.Data,
    BloqueActual^.Nonce,
    BloqueActual^.PreviousHash
);

if HashCalculado <> BloqueActual^.Hash then
begin
    WriteLn( 'ERROR: -Hash- inv lido -en- bloque- ', BloqueActual^.Index );
    Result := False;
    Exit;
end;

BloqueActual := BloqueActual^.Siguiente;
end;

WriteLn( ' -Blockchain- ntegro ' );
end;

```

11. Implementación del Árbol de Merkle

11.1. Conceptos del Árbol de Merkle

El Árbol de Merkle permite verificar la integridad de un conjunto de datos mediante hashing jerárquico. En EDDMail, reemplaza al Árbol B para gestionar favoritos con verificación criptográfica.

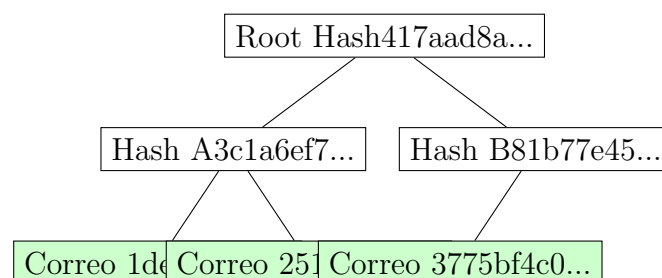


Figura 6: Estructura del Árbol de Merkle

11.2. Generación de Hash SHA-256

Listing 26: Función hash SHA-256 simulada

```
function TEDDMailSystem.GenerarHashSHA256(Datos: String): String;  
var  
    i: Integer;  
    HashVal1, HashVal2: Cardinal;  
    Temp: String;  
begin  
    // Algoritmo DJB2 combinado para simular SHA-256  
    HashVal1 := 5381;  
    HashVal2 := 0;  
  
    for i := 1 to Length(Datos) do  
    begin  
        HashVal1 := ((HashVal1 shl 5) + HashVal1) + Ord(Datos[i]);  
        HashVal2 := HashVal2 xor (Ord(Datos[i]) * 31);  
    end;  
  
    // Generar string hexadecimal de 64 caracteres  
    Temp := IntToHex(HashVal1, 8) + IntToHex(HashVal2, 8);  
  
    // Extender a 64 caracteres (simular SHA-256)  
    while Length(Temp) < 64 do  
        Temp := Temp + IntToHex(HashVal1 xor HashVal2, 8);  
  
    Result := Copy(Temp, 1, 64);  
end;
```

11.3. Construcción del Árbol

Listing 27: Construir árbol de Merkle desde favoritos

```
procedure TEDDMailSystem.ConstruirArbolMerkleDesdeCorreos(  
    Usuario: PUsuario);  
var  
    ListaFavoritos: TStringList;  
    ListaHojas: array of PNodeMerkle;  
    i, CorreoId: Integer;  
    Correo: PCorreo;  
    NodoHoja: PNodeMerkle;  
    DatosCorreo: String;  
begin  
    if Usuario = nil then Exit;  
  
    // Liberar rbol anterior  
    if Usuario^.ArbolMerkleFavoritos <> nil then  
    begin
```

```

    LiberarArbolMerkle(Usuario^.ArbolMerkleFavoritos);
    Usuario^.ArbolMerkleFavoritos := nil;
end;

// Recolectar IDs de favoritos
ListaFavoritos := TStringList.Create;
try
    RecorrerFavoritosMerkle(Usuario, ListaFavoritos);

    if ListaFavoritos.Count = 0 then Exit;

    // Crear array de hojas
    SetLength(ListaHojas, ListaFavoritos.Count);

    for i := 0 to ListaFavoritos.Count - 1 do
    begin
        CorreoId := StrToIntDef(ListaFavoritos[i], -1);
        if CorreoId = -1 then Continue;

        Correo := BuscarCorreoEnBandeja(Usuario, CorreoId);
        if Correo = nil then Continue;

        // Crear nodo hoja
        New(NodoHoja);
        NodoHoja^.EsHoja := True;
        NodoHoja^.Correo := Correo;
        NodoHoja^.Izquierdo := nil;
        NodoHoja^.Derecho := nil;

        // Generar hash de los datos del correo
        DatosCorreo := IntToStr(Correo^.Id) +
            Correo^.Remitente +
            Correo^.Asunto +
            Correo^.Fecha +
            Correo^.Mensaje;
        NodoHoja^.Hash := GenerarHashSHA256(DatosCorreo);

        ListaHojas[i] := NodoHoja;
    end;

    // Construir rbol balanceado
    Usuario^.ArbolMerkleFavoritos :=
        ConstruirArbolMerkleBalanceado(ListaHojas);

    WriteLn('      - rbol -de-Merkle-construido-con-',
        ListaFavoritos.Count, ' -hojas');

finally

```

```

    ListaFavoritos.Free;
end;
end;

```

11.4. Combinar Hashes

Listing 28: Combinar hashes de nodos hijos

```

function TEDDMailSystem.CombinarHashesMerkle(HashIzq ,
    HashDer: String): String;
var
    Concatenado: String;
begin
    if HashDer = '' then
        Result := HashIzq // Nodo impar, duplicar hash
    else
        begin
            Concatenado := HashIzq + HashDer;
            Result := GenerarHashSHA256(Concatenado);
        end;
    end;

function TEDDMailSystem.ConstruirArbolMerkleBalanceado(
    ListaHojas: array of PNodeMerkle): PNodeMerkle;
var
    Nivel: array of PNodeMerkle;
    SiguienteNivel: array of PNodeMerkle;
    i, Tam: Integer;
    NodoPadre: PNodeMerkle;
begin
    if Length(ListaHojas) = 0 then
        begin
            Result := nil;
            Exit;
        end;

    if Length(ListaHojas) = 1 then
        begin
            Result := ListaHojas[0];
            Exit;
        end;

    // Inicializar nivel actual
    SetLength(Nivel, Length(ListaHojas));
    for i := 0 to High(ListaHojas) do
        Nivel[i] := ListaHojas[i];

    // Construir rbol nivel por nivel

```

```

while Length(Nivel) > 1 do
begin
  Tam := (Length(Nivel) + 1) div 2;
  SetLength(SiguienteNivel, Tam);

  for i := 0 to Tam - 1 do
    begin
      New(NodoPadre);
      NodoPadre^.EsHoja := False;
      NodoPadre^.Correo := nil;

      NodoPadre^.Izquierdo := Nivel[i * 2];

      if (i * 2 + 1) < Length(Nivel) then
        NodoPadre^.Derecho := Nivel[i * 2 + 1]
      else
        NodoPadre^.Derecho := nil;

      NodoPadre^.Hash := CombinarHashesMerkle(
        NodoPadre^.Izquierdo^.Hash,
        IfThen(NodoPadre^.Derecho <> nil,
          NodoPadre^.Derecho^.Hash, '' )
      );

      SiguienteNivel[i] := NodoPadre;
    end;

  Nivel := SiguienteNivel;
end;

  Result := Nivel[0];
end;

```

11.5. Verificación de Integridad

Listing 29: Verificar integridad con Merkle

```

function TEDDMailSystem.VerificarIntegridadMerkle(
  Usuario: PUsuario): Boolean;
var
  HashRaizRecalculado: String;

  function RecalcularHashRecursivo(Nodo: PNodeMerkle): String;
  var
    DatosCorreo: String;
  begin
    if Nodo = nil then
      begin

```

```

    Result := '';
    Exit;
end;

if Nodo^.EsHoja then
begin
    // Recalcular hash de la hoja
    DatosCorreo := IntToStr(Nodo^.Correo^.Id) +
        Nodo^.Correo^.Remitente +
        Nodo^.Correo^.Asunto +
        Nodo^.Correo^.Fecha +
        Nodo^.Correo^.Mensaje;
    Result := GenerarHashSHA256(DatosCorreo);
end
else
begin
    // Combinar hashes de hijos
    Result := CombinarHashesMerkle(
        RecalcularHashRecursivo(Nodo^.Izquierdo),
        RecalcularHashRecursivo(Nodo^.Derecho)
    );
end;
end;

begin
    Result := False;

    if (Usuario = nil) or (Usuario^.ArbolMerkleFavoritos = nil) then
    begin
        WriteLn('No hay árbol de Merkle para verificar');
        Exit;
    end;

    // Recalcular hash raíz
    HashRaizRecalculado :=
        RecalcularHashRecursivo(Usuario^.ArbolMerkleFavoritos);

    // Comparar con hash almacenado
    if HashRaizRecalculado = Usuario^.ArbolMerkleFavoritos^.Hash then
    begin
        WriteLn('Integridad de favoritos verificada');
        Result := True;
    end
    else
    begin
        WriteLn('ADVERTENCIA: Favoritos alterados');
        WriteLn('Hash esperado: ', Usuario^.ArbolMerkleFavoritos^.Hash);
        WriteLn('Hash recalculado: ', HashRaizRecalculado);
    end;
end;

```

```

    Result := False;
end;
end;

```

12. Implementación de Compresión LZW

12.1. Algoritmo Lempel-Ziv-Welch

El algoritmo LZW es un método de compresión sin pérdida que construye un diccionario dinámico de secuencias.

Listing 30: Compresión LZW

```

function TEDDMailSystem.ComprimirLZW(Texto: String): String;
var
    Diccionario: TStringList;
    i,CodigoActual: Integer;
    Actual, Siguiente: String;
    Resultado: TStringList;
begin
    Resultado := TStringList.Create;
    Diccionario := TStringList.Create;
    try
        // Inicializar diccionario con caracteres ASCII (0–255)
        for i := 0 to 255 do
            Diccionario.Add(Chr(i));

        CodigoActual := 256;
        Actual := '';

        for i := 1 to Length(Texto) do
            begin
                Siguiente := Actual + Texto[i];

                if Diccionario.IndexOf(Siguiente) >= 0 then
                    begin
                        // Secuencia existe en diccionario
                        Actual := Siguiente;
                    end
                else
                    begin
                        // Emitir código de la secuencia actual
                        Resultado.Add(IntToStr(Diccionario.IndexOf(Actual)));

                        // Agregar nueva secuencia al diccionario
                        Diccionario.Add(Siguiente);
                        Inc(CodigoActual);

                        Actual := Texto[i];
                    end
                end
            end

```



```

    end;
end;

// Emitir ltimo c digo
if Actual <> '' then
    Resultado.Add(IntToStr(Diccionario.IndexOf(Actual)));

Result := Resultado.CommaText;

finally
    Diccionario.Free;
    Resultado.Free;
end;
end;

```

12.2. Descargar Correo Comprimido

Listing 31: Guardar correo comprimido

```

function TEDDMailSystem.GuardarCorreoComprimido(Usuario: PUsuario;
    CorreoId: Integer; RutaArchivo: String): Boolean;
var
    Correo: PCorreo;
    ContenidoCompleto, Comprimido: String;
    TamOriginal, TamComprimido: Integer;
    Ratio: Double;
begin
    Result := False;

    // Buscar correo
    Correo := BuscarCorreoEnBandeja(Usuario, CorreoId);
    if Correo = nil then Exit;

    // Construir contenido completo
    ContenidoCompleto := Format(
        '====CORREO-COMPRIMIDO--LZW====' + LineEnding +
        'ID: %d' + LineEnding +
        'Remitente: %s' + LineEnding +
        'Destinatario: %s' + LineEnding +
        'Asunto: %s' + LineEnding +
        'Fecha: %s' + LineEnding + LineEnding +
        '====MENSAJE-COMPRIMIDO====' + LineEnding +
        '%s' + LineEnding + LineEnding +
        '====INFORMACIN====' + LineEnding +
        'Tama o original: %d-bytes' + LineEnding +
        'Tama o comprimido: %d-bytes' + LineEnding +
        'Ratio-de-compresi n: %.2f%%',
        [Correo^.Id,

```

```

    Correo^.Remitente,
    Correo^.Destinatario,
    Correo^.Asunto,
    Correo^.Fecha,
    '%s', // Placeholder para mensaje comprimido
    0, 0, 0.0]); // Placeholders para tama os

// Comprimir mensaje
Comprimido := ComprimirLZW(Correo^.Mensaje);

// Calcular m tricas
TamOriginal := Length(Correo^.Mensaje);
TamComprimido := Length(Comprimido);
Ratio := ((TamOriginal - TamComprimido) / TamOriginal) * 100;

// Construir contenido final
ContenidoCompleto := Format(
    ContenidoCompleto,
    [Comprimido, TamOriginal, TamComprimido, Ratio]);

// Guardar archivo
Result := GuardarArchivoTexto(RutaArchivo, ContenidoCompleto);

if Result then
    WriteLn('Correo-comprimido-guardado:', RutaArchivo)
else
    WriteLn('Error-al-guardar-correo-comprimido');
end;

```

13. Implementación de Control de Logueo

13.1. Registro de Sesiones

Listing 32: Registrar entrada de usuario

```

procedure TEDDMailSystem.RegistrarEntrada(Email: String);
var
    NuevoRegistro: PRegistroLogueo;
begin
    New(NuevoRegistro);
    NuevoRegistro^.Usuario := Email;
    NuevoRegistro^.Entrada := Now;
    NuevoRegistro^.Salida := 0; // Pendiente

    // Agregar al inicio de la lista
    NuevoRegistro^.Siguiete := FListaLogueo;
    FListaLogueo := NuevoRegistro;

```

```

    WriteLn( 'Entrada-registrada:-', Email);
end;

procedure TEDDMailSystem.RegistrarSalida(Email: String);
var
    Registro: PRegistroLogueo;
begin
    Registro := FListaLogueo;

    // Buscar registro activo (sin salida)
    while Registro <> nil do
    begin
        if (Registro^.Usuario = Email) and (Registro^.Salida = 0) then
        begin
            Registro^.Salida := Now;
            WriteLn( 'Salida-registrada:-', Email);
            Exit;
        end;
        Registro := Registro^.Siguiente;
    end;

    WriteLn( 'Advertencia:-No-se-encontró-sesión-activa-para-', Email);
end;

```

13.2. Obtener Logs

Listing 33: Obtener logs de logueo

```

function TEDDMailSystem.ObtenerLogsDeLogueo: TStringList;
var
    Registro: PRegistroLogueo;
    Duracion: TDateTime;
    DuracionStr: String;
begin
    Result := TStringList.Create;

    Registro := FListaLogueo;
    while Registro <> nil do
    begin
        if Registro^.Salida > 0 then
        begin
            Duracion := Registro^.Salida - Registro^.Entrada;
            DuracionStr := FormatDateTime( 'hh:nn:ss ', Duracion);
        end
        else
            DuracionStr := 'Sesión-activa';

        Result.Add(Format(

```

```

    'Usuario: -%s' + LineEnding +
    'Entrada: -%s' + LineEnding +
    'Salida: -%s' + LineEnding +
    'Duraci n: -%s' + LineEnding +
    ,
    [Registro^.Usuario ,
    FormatDateTime( 'dd/mm/yyyy-hh:nn:ss ', Registro^.Entrada),
    IfThen( Registro^.Salida > 0,
            FormatDateTime( 'dd/mm/yyyy-hh:nn:ss ', Registro^.Salida),
            'Pendiente '),
    DuracionStr ]));

    Registro := Registro^.Siguiente;
end;

if Result.Count = 0 then
    Result.Add( 'No-hay-registros-de-logueo ');
end;

```

13.3. Exportación JSON

Listing 34: Exportar logueo a JSON

```

function TEDDMailSystem.ExportarLogueoJSON(
    RutaArchivo: String): Boolean;
var
    JSON: TJSONObject;
    JSONArray: TJSONArray;
    JSONLogueo: TJSONObject;
    Registro: PRegistroLogueo;
    Archivo: TStringList;
begin
    Result := False;

    try
        JSON := TJSONObject.Create;
        JSONArray := TJSONArray.Create;

        Registro := FListaLogueo;
        while Registro <> nil do
            begin
                JSONLogueo := TJSONObject.Create;
                JSONLogueo.Add( 'usuario ', Registro^.Usuario);
                JSONLogueo.Add( 'entrada ',
                    FormatDateTime( 'yyyy-mm-dd"T"hh:nn:ss ', Registro^.Entrada));

                if Registro^.Salida > 0 then
                    JSONLogueo.Add( 'salida ',

```

```

        FormatDateTime( 'yyyy-mm-dd"T"hh:nn:ss ', Registro^.Salida))
    else
        JSONLogueo.Add( 'salida ', 'activa ');

        JSONArray.Add(JSONLogueo);
        Registro := Registro^.Siguiente;
    end;

JSON.Add( 'logueos ', JSONArray);
JSON.Add( 'total_registros ', JSONArray.Count);
JSON.Add( 'fecha_exportacion ',
    FormatDateTime( 'yyyy-mm-dd"T"hh:nn:ss ', Now));

// Guardar archivo
Archivo := TStringList.Create;
try
    Archivo.Text := JSON.FormatJSON;
    Archivo.SaveToFile( RutaArchivo);
    Result := True;
finally
    Archivo.Free;
end;

finally
    JSON.Free;
end;
end;

```

14. Implementación de Grafo de Contactos

14.1. Estructura del Grafo

Listing 35: Tipos para grafo de contactos

```

type
    PNodeGrafo = ^TNodeGrafo;
    PAristaGrafo = ^TAristaGrafo;

    TAristaGrafo = record
        Destino: PNodeGrafo;
        Siguiente: PAristaGrafo;
    end;

    TNodeGrafo = record
        Id: Integer;
        Nombre: String;
        Usuario: String;
        Email: String;
    end;

```

```

    ListaAdyacencia: PAristaGrafo;
    Siguiente: PNodeGrafo;
end;

```

14.2. Carga Masiva desde JSON

Listing 36: Cargar contactos desde JSON

```

procedure TEDDMailSystem.CargarContactosDesdeJSON(
    RutaArchivo: String);
var
    JSON: TJSONData;
    JSONArray: TJJSONArray;
    JSONUsuario, JSONContacto: TJSONObject;
    i, j: Integer;
    EmailUsuario, EmailContacto: String;
    Usuario: PUsuario;
begin
    try
        JSON := GetJSON(TFileStream.Create(RutaArchivo, fmOpenRead));

        JSONArray := TJJSONArray(JSON.FindPath('usuarios'));
        if JSONArray = nil then
            begin
                WriteLn('Error: ~Formato~JSON~inv~lido~');
                Exit;
            end;

        // Procesar cada usuario
        for i := 0 to JSONArray.Count - 1 do
            begin
                JSONUsuario := TJSONObject(JSONArray[i]);
                EmailUsuario := JSONUsuario.Get('email', '');

                Usuario := BuscarUsuario(EmailUsuario);
                if Usuario = nil then
                    begin
                        WriteLn('Advertencia: ~Usuario~no~existe~', EmailUsuario);
                        Continue;
                    end;

                // Cargar contactos del usuario
                JSONContacto := TJSONObject(JSONUsuario.FindPath('contactos'));
                if JSONContacto <> nil then
                    begin
                        for j := 0 to TJJSONArray(JSONContacto).Count - 1 do
                            begin
                                EmailContacto := TJJSONArray(JSONContacto).Strings[j];

```

```

        if BuscarUsuario(EmailContacto) <> nil then
        begin
            AgregarContacto(Usuario, EmailContacto);
            WriteLn('Contacto agregado:', EmailUsuario,
                    ' - ', EmailContacto);
        end
        else
            WriteLn('Advertencia: -Contacto no existe:-', EmailContacto);
        end;
    end;
end;

WriteLn('      -Carga masiva completada');

finally
    JSON.Free;
end;
end;

```

14.3. Generar Reporte Graphviz

Listing 37: Reporte de grafo de contactos

```

procedure TEDDMailSystem.GenerarReporteGrafoContactos(
    RutaSalida: String);
var
    Archivo: TextFile;
    Process: TProcess;
    NombreArchivo: String;
    Usuario: PUsuario;
    Contacto: PContacto;
    Visitados: TStringList;
begin
    try
        ForceDirectories(RutaSalida);
        NombreArchivo := RutaSalida + '/grafo_contactos.dot';

        AssignFile(Archivo, NombreArchivo);
        Rewrite(Archivo);

        WriteLn(Archivo, 'graph-G{');
        WriteLn(Archivo, '----label="Grafo de Contactos -- Usuarios y Relaciones');
        WriteLn(Archivo, '----fontsize=16;');
        WriteLn(Archivo, '----node-[shape=circle, style=filled];');

        Visitados := TStringList.Create;
    try

```

```

Usuario := FUsuarios;
while Usuario <> nil do
begin
    // Nodo del usuario
    WriteLn(Archivo, Format('-----%s'~[label=%s\nID:~%d",~fillcolor=li
        [Usuario^.Email, Usuario^.Usuario, Usuario^.Id]));

    // Aristas a contactos
    Contacto := Usuario^.ListaContactos;
    while Contacto <> nil do
    begin
        // Evitar aristas duplicadas
        if Visitados.IndexOf(Usuario^.Email + '-' + Contacto^.Email) < 0
        begin
            WriteLn(Archivo, Format('-----%s'~----%s";',
                [Usuario^.Email, Contacto^.Email]));

            Visitados.Add(Usuario^.Email + '-' + Contacto^.Email);
            Visitados.Add(Contacto^.Email + '-' + Usuario^.Email);
        end;

        Contacto := Contacto^.Siguiente;
        if Contacto = Usuario^.ListaContactos then Break; // Lista circul
    end;

    Usuario := Usuario^.Siguiente;
end;

finally
    Visitados.Free;
end;

WriteLn(Archivo, '}');
CloseFile(Archivo);

// Generar imagen
try
    Process := TProcess.Create(nil);
    try
        Process.Executable := 'dot';
        Process.Parameters.Add('-Tpng');
        Process.Parameters.Add(NombreArchivo);
        Process.Parameters.Add('-o');
        Process.Parameters.Add(ChangeFileExt(NombreArchivo, '.png'));
        Process.Options := Process.Options + [poWaitOnExit];
        Process.Execute;
        WriteLn('Reporte~de~grafo~generado:~',
            ChangeFileExt(NombreArchivo, '.png'));
    finally
        Process.Free;
    end;
end;

```



```

        finally
            Process.Free;
        end;
    except
        on E: Exception do
            WriteLn( 'Error al generar imagen: ', E.Message );
        end;

    except
        on E: Exception do
            WriteLn( 'Error al generar reporte de grafo: ', E.Message );
        end;
    end;
end;

```

15. Integración con Interfaz GTK

15.1. Ventana de Borradores

Listing 38: Interfaz para gestión de borradores

```

procedure TInterfazEDDMail.CrearVentanaBorradores;
var
    FormBorradores: TForm;
    PanelTop, PanelBottom: TPanel;
    ListBox: TListBox;
    BtnNuevo, BtnEditar, BtnEnviar, BtnEliminar: TButton;
    ComboRecorrido: TComboBox;
    LabelRecorrido: TLabel;
begin
    FormBorradores := TForm.Create( nil );
    try
        with FormBorradores do
            begin
                Caption := 'Borradores ( rbol AVL )';
                Width := 600;
                Height := 500;
                Position := poScreenCenter;
            end;

            // Panel superior con controles
            PanelTop := TPanel.Create( FormBorradores );
            with PanelTop do
                begin
                    Parent := FormBorradores;
                    Align := alTop;
                    Height := 60;
                    BevelOuter := bvNone;
                end;

```

```

LabelRecorrido := TLabel.Create(PanelTop);
with LabelRecorrido do
begin
    Parent := PanelTop;
    Caption := 'Tipo-de-recorrido: ';
    Left := 10;
    Top := 20;
end;

ComboRecorrido := TComboBox.Create(PanelTop);
with ComboRecorrido do
begin
    Parent := PanelTop;
    Left := 120;
    Top := 15;
    Width := 150;
    Items.Add( 'InOrden-(LRD)' );
    Items.Add( 'PreOrden-(RLD)' );
    Items.Add( 'PostOrden-(LDR)' );
    ItemIndex := 0;
    OnChange := @OnRecorridoChange;
end;

// ListBox para mostrar borradores
ListBox := TListBox.Create(FormBorradores);
with ListBox do
begin
    Parent := FormBorradores;
    Align := alClient;
end;

// Panel inferior con botones
PanelBottom := TPanel.Create(FormBorradores);
with PanelBottom do
begin
    Parent := FormBorradores;
    Align := alBottom;
    Height := 60;
    BevelOuter := bvNone;
end;

BtnNuevo := TButton.Create(PanelBottom);
with BtnNuevo do
begin
    Parent := PanelBottom;
    Caption := 'Nuevo-Borrador';
    Left := 10;

```

```

    Top := 15;
    Width := 130;
    OnClick := @OnNuevoBorradorClick;
end;

BtnEditar := TButton.Create(PanelBottom);
with BtnEditar do
begin
    Parent := PanelBottom;
    Caption := 'Editar';
    Left := 150;
    Top := 15;
    Width := 100;
    OnClick := @OnEditarBorradorClick;
end;

BtnEnviar := TButton.Create(PanelBottom);
with BtnEnviar do
begin
    Parent := PanelBottom;
    Caption := 'Enviar';
    Left := 260;
    Top := 15;
    Width := 100;
    OnClick := @OnEnviarBorradorClick;
end;

BtnEliminar := TButton.Create(PanelBottom);
with BtnEliminar do
begin
    Parent := PanelBottom;
    Caption := 'Eliminar';
    Left := 370;
    Top := 15;
    Width := 100;
    OnClick := @OnEliminarBorradorClick;
end;

// Cargar borradores iniciales
CargarBorradores(ListBox, 'InOrden');

FormBorradores.ShowModal;
finally
    FormBorradores.Free;
end;
end;

```

15.2. Ventana de Comunidades

Listing 39: Interfaz para comunidades BST

```
procedure TInterfazEDDMail.CrearVentanaComunidades;  
var  
    FormComunidades: TForm;  
    TreeView: TTreeView;  
    PanelBotones: TPanel;  
    BtnCrear, BtnPublicar, BtnVerMensajes: TButton;  
begin  
    FormComunidades := TForm.Create(nil);  
    try  
        with FormComunidades do  
            begin  
                Caption := 'Comunidades-( rbol -BST)';  
                Width := 500;  
                Height := 600;  
                Position := poScreenCenter;  
            end;  
  
            // TreeView para mostrar jerarquía  
            TreeView := TTreeView.Create(FormComunidades);  
            with TreeView do  
                begin  
                    Parent := FormComunidades;  
                    Align := alClient;  
                end;  
  
            // Panel de botones  
            PanelBotones := TPanel.Create(FormComunidades);  
            with PanelBotones do  
                begin  
                    Parent := FormComunidades;  
                    Align := alBottom;  
                    Height := 60;  
                end;  
  
            BtnCrear := TButton.Create(PanelBotones);  
            with BtnCrear do  
                begin  
                    Parent := PanelBotones;  
                    Caption := 'Crear-Comunidad';  
                    Left := 10;  
                    Top := 15;  
                    Width := 140;  
                    OnClick := @OnCrearComunidadClick;  
                end;
```

```

    BtnPublicar := TButton.Create(PanelBotones);
with BtnPublicar do
begin
    Parent := PanelBotones;
    Caption := 'Publicar Mensaje';
    Left := 160;
    Top := 15;
    Width := 140;
    OnClick := @OnPublicarMensajeClick;
end;

    BtnVerMensajes := TButton.Create(PanelBotones);
with BtnVerMensajes do
begin
    Parent := PanelBotones;
    Caption := 'Ver Mensajes';
    Left := 310;
    Top := 15;
    Width := 140;
    OnClick := @OnVerMensajesClick;
end;

    // Cargar comunidades en TreeView
    CargarComunidadesEnTree(TreeView);

    FormComunidades.ShowModal;
finally
    FormComunidades.Free;
end;
end;

```

16. Análisis de Complejidad - Proyecto Completo

16.1. Complejidades Temporales de Todas las Estructuras

Operación	Estructura	Mejor Caso	Peor Caso
Fase 1			
Insertar	Lista Simple	$O(1)$	$O(1)$
Buscar	Lista Simple	$O(1)$	$O(n)$
Eliminar	Lista Simple	$O(1)$	$O(n)$
Push	Pila	$O(1)$	$O(1)$
Pop	Pila	$O(1)$	$O(1)$
Enqueue	Cola	$O(1)$	$O(1)$
Dequeue	Cola	$O(1)$	$O(1)$
Fase 2			
Insertar	Árbol AVL	$O(\log n)$	$O(\log n)$
Buscar	Árbol AVL	$O(\log n)$	$O(\log n)$
Eliminar	Árbol AVL	$O(\log n)$	$O(\log n)$
Insertar	Árbol BST	$O(\log n)$	$O(n)$
Buscar	Árbol BST	$O(\log n)$	$O(n)$
Insertar	Árbol B	$O(\log n)$	$O(\log n)$
Buscar	Árbol B	$O(\log n)$	$O(\log n)$
Fase 3			
Agregar Bloque	Blockchain	$O(n)^*$	$O(n)^*$
Verificar	Blockchain	$O(n)$	$O(n)$
Construir	Merkle Tree	$O(n)$	$O(n)$
Verificar	Merkle Tree	$O(\log n)$	$O(\log n)$
Buscar	Merkle Tree	$O(\log n)$	$O(\log n)$
Comprimir	LZW	$O(n)$	$O(n)$
Descomprimir	LZW	$O(n)$	$O(n)$
Registrar	Logueo	$O(1)$	$O(1)$
Buscar	Logueo	$O(1)$	$O(n)$
Agregar Arista	Grafo	$O(1)$	$O(1)$
Buscar Nodo	Grafo	$O(1)$	$O(V)$
Recorrer	Grafo	$O(V+E)$	$O(V+E)$

Cuadro 2: Complejidades temporales - Proyecto completo

** Incluye tiempo de minería (proof-of-work)*

16.2. Ventajas de Cada Estructura

16.2.1. Árbol AVL

■ Ventajas:

- Garantiza balanceo estricto (factor ≤ 1)
- Búsquedas muy rápidas y predecibles

- Ideal para aplicaciones con muchas consultas
- **Desventajas:**
 - Más rotaciones durante inserción/eliminación
 - Overhead de mantener factor de balance
- **Uso en EDDMail:** Borradores que requieren acceso ordenado frecuente

16.2.2. Árbol BST

- **Ventajas:**
 - Implementación simple
 - Bajo overhead de memoria
 - Inserción rápida sin rotaciones
- **Desventajas:**
 - Puede degenerar en lista (sin balanceo)
 - Rendimiento variable según orden de inserción
- **Uso en EDDMail:** Comunidades con nombres naturalmente distribuidos

16.2.3. Árbol B

- **Ventajas:**
 - Excelente para grandes volúmenes de datos
 - Minimiza accesos a disco
 - Nodos con múltiples claves reducen altura
- **Desventajas:**
 - Implementación compleja
 - Mayor uso de memoria por nodo
- **Uso en EDDMail:** Favoritos con acceso rápido por ID

17. Diagramas de Flujo - Fase 2

17.1. Flujo de Guardado de Borrador

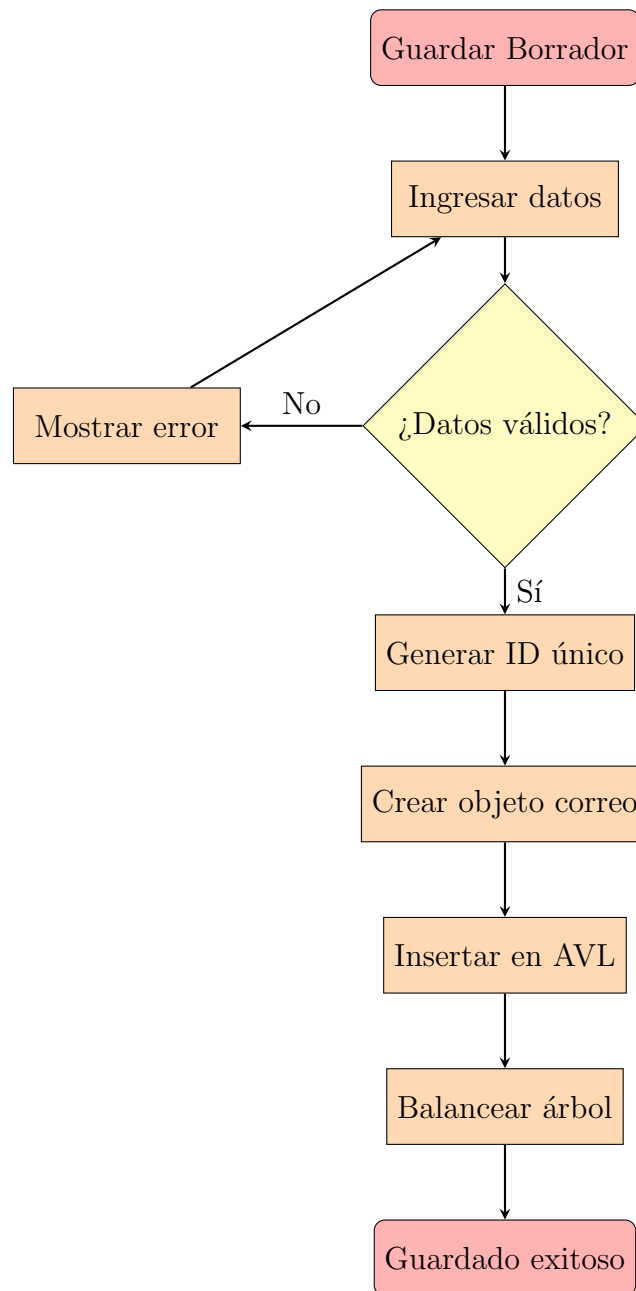


Figura 7: Diagrama de flujo - Guardar borrador en AVL

17.2. Flujo de Creación de Comunidad

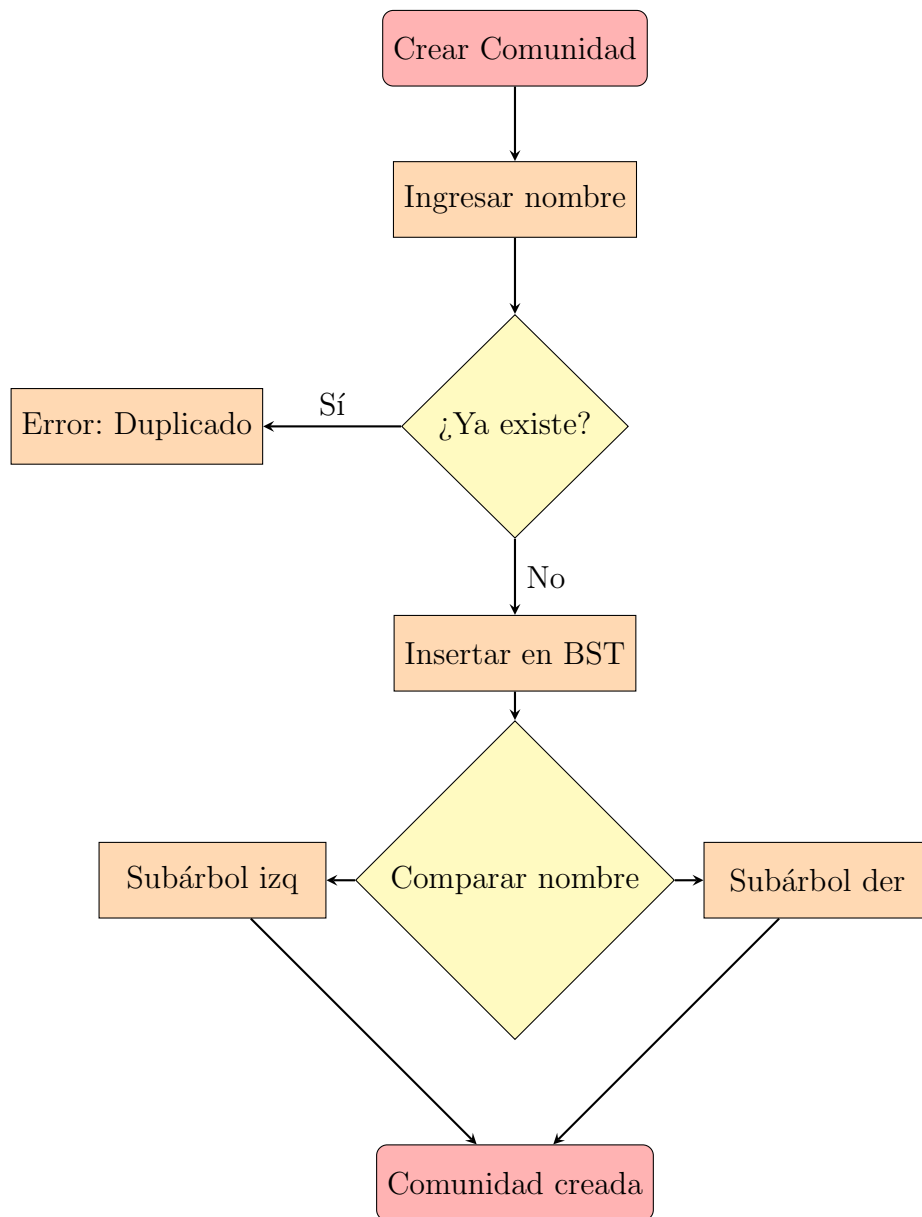


Figura 8: Diagrama de flujo - Crear comunidad en BST

17.3. Flujo de Inserción en Árbol B

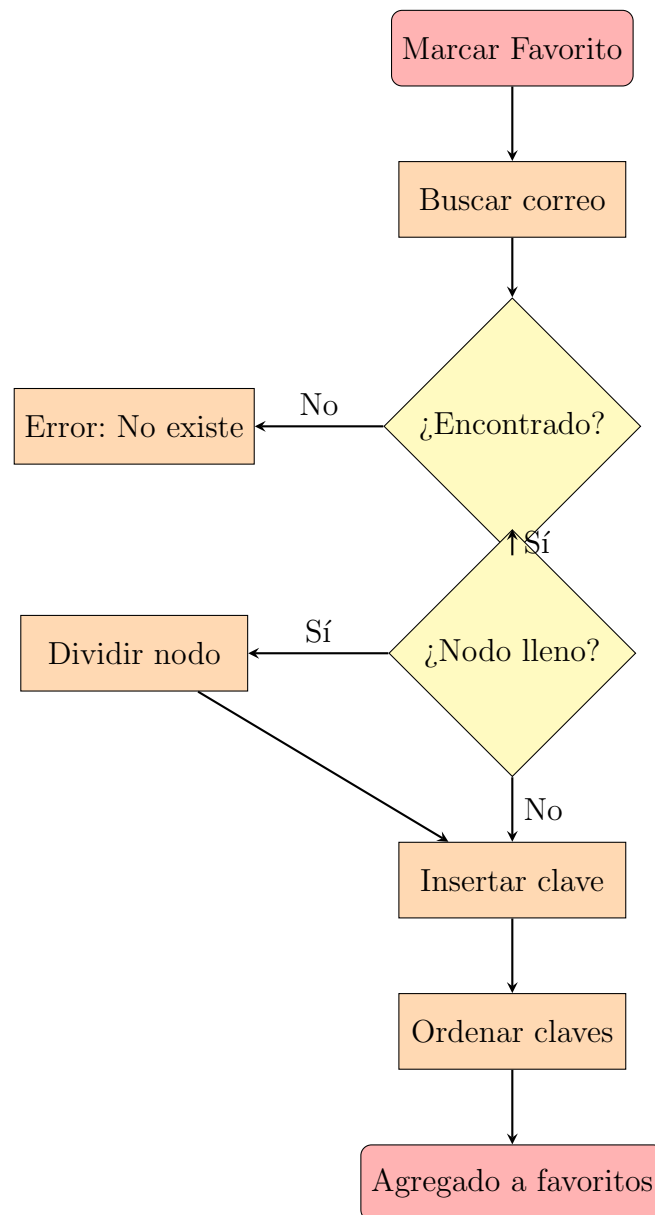


Figura 9: Diagrama de flujo - Inserción en Árbol B

18. Formato JSON para Carga Masiva

18.1. Estructura JSON de Usuarios

Listing 40: Formato JSON usuarios - Fase 2

```
1 {
2   "usuarios": [
3     {
4       "id": 1,
5       "nombre": "Luis Garcia",
```

```

6      "usuario": "auxluis",
7      "password": "auxluis123",
8      "email": "aux-luis@edd.com",
9      "telefono": "12345678"
10   },
11   {
12       "id": 2,
13       "nombre": "Marcos Itzep",
14       "usuario": "auxmarcos",
15       "password": "auxmarcos123",
16       "email": "aux-marcosg@edd.com",
17       "telefono": "87654321"
18   }
19 ]
20 }

```

18.2. Estructura JSON de Correos

Listing 41: Formato JSON correos - Fase 2

```

1 {
2   "correos": [
3     {
4       "id": 1,
5       "remite nte": "aux-luis@edd.com",
6       "destinatario": "aux-marcosg@edd.com",
7       "estado": "NL",
8       "asunto": "Reuni n de proyecto",
9       "mensaje": "Confirmar asistencia a la reuni n de
10         equipo."
11     },
12     {
13       "id": 2,
14       "remite nte": "aux-luis@edd.com",
15       "destinatario": "aux-marcosg@edd.com",
16       "estado": "NL",
17       "asunto": "Reporte semanal",
18       "mensaje": "Adjunto el reporte semanal de actividades."
19     }
20   ]
21 }

```

19. Testing y Validaci3n

19.1. Pruebas del 1rbol AVL

Listing 42: Validación de balanceo AVL

```

function TEDDMailSystem.ValidarBalanceoAVL(nodo: PNodeAVL): Boolean;
var
    Balance: Integer;
begin
    Result := True;
    if nodo = nil then Exit;

    // Verificar balance del nodo actual
    Balance := ObtenerBalance(nodo);
    if (Balance < -1) or (Balance > 1) then
    begin
        WriteLn('ERROR: -Nodo- desbalanceado -con-ID:- ', nodo^.Correo^.Id);
        WriteLn('Factor -de- balance:- ', Balance);
        Result := False;
        Exit;
    end;

    // Verificar recursivamente sub rboles
    if not ValidarBalanceoAVL(nodo^.Izquierdo) then
        Result := False;

    if not ValidarBalanceoAVL(nodo^.Derecho) then
        Result := False;
end;

```

19.2. Pruebas del Árbol BST

Listing 43: Validación de orden BST

```

function TEDDMailSystem.ValidarOrdenBST(nodo: PNodeBST;
    var anterior: String): Boolean;
begin
    Result := True;
    if nodo = nil then Exit;

    // Verificar sub rbol izquierdo
    if not ValidarOrdenBST(nodo^.Izquierdo, anterior) then
    begin
        Result := False;
        Exit;
    end;

    // Verificar orden actual
    if (anterior < ' ') and
        (CompareText(anterior, nodo^.NombreComunidad) >= 0) then
    begin
        WriteLn('ERROR: -BST- violado -entre- " ', anterior,

```

```

        '-'y-' ', nodo^.NombreComunidad, ' ');
    Result := False;
    Exit;
end;

anterior := nodo^.NombreComunidad;

// Verificar sub rbol derecho
if not ValidarOrdenBST(nodo^.Derecho, anterior) then
    Result := False;
end;

```

19.3. Pruebas del Árbol B

Listing 44: Validación de propiedades Árbol B

```

function TEDDMailSystem.ValidarPropiedadesB(nodo: PNodeB): Boolean;
var
    i: Integer;
begin
    Result := True;
    if nodo = nil then Exit;

    // Verificar n mero de claves (entre 2 y 4, excepto ra z)
    if (nodo^.NumClaves < 1) or (nodo^.NumClaves > 4) then
    begin
        WriteLn('ERROR: -N mero -inv lido -de -claves: -', nodo^.NumClaves);
        Result := False;
        Exit;
    end;

    // Verificar orden de claves
    for i := 0 to nodo^.NumClaves - 2 do
    begin
        if nodo^.Claves[i] >= nodo^.Claves[i + 1] then
        begin
            WriteLn('ERROR: -Claves -desordenadas -en -nodo');
            Result := False;
            Exit;
        end;
    end;

    // Verificar hijos recursivamente
    if not nodo^.EsHoja then
    begin
        for i := 0 to nodo^.NumClaves do
        begin
            if nodo^.Hijos[i] <> nil then

```

```

begin
  if not ValidarPropiedadesB(nodo^.Hijos[i]) then
    Result := False;
  end;
end;
end;
end;
end;

```

20. Casos de Uso - Fase 2

20.1. Caso de Uso: Gestión de Borradores

Campo	Descripción
Nombre	Guardar y editar borradores
Actor	Usuario registrado
Precondiciones	Usuario autenticado
Flujo Principal	1. Usuario selecciona "Nuevo Borrador" 2. Ingresa destinatario, asunto y mensaje 3. Sistema guarda en árbol AVL 4. Sistema balancea el árbol 5. Confirma guardado exitoso
Postcondiciones	Borrador almacenado en AVL balanceado
Flujo Alternativo	3a. Datos inválidos → Mostrar error 4a. Usuario edita borrador existente 4b. Usuario envía borrador

Cuadro 3: Caso de uso - Borradores

20.2. Caso de Uso: Comunidades

Campo	Descripción
Nombre	Crear y gestionar comunidades
Actor	Usuario root o usuario registrado
Precondiciones	Sistema iniciado
Flujo Principal	1. Root crea comunidad con nombre 2. Sistema inserta en árbol BST 3. Usuario publica mensaje en comunidad 4. Sistema agrega a lista de mensajes 5. Incrementa contador de mensajes
Postcondiciones	Comunidad activa con mensajes
Flujo Alternativo	2a. Comunidad duplicada → Error 3a. Comunidad no existe → Error

Cuadro 4: Caso de uso - Comunidades

21. Mejores Prácticas de Implementación

21.1. Manejo de Memoria en Árboles

Listing 45: Liberación correcta de memoria

```
procedure TEDDMailSystem.LiberarArbolAVL(var raiz: PNodeAVL);  
begin  
    if raiz = nil then Exit;  
  
    // Liberar recursivamente (PostOrden)  
    LiberarArbolAVL(raiz^.Izquierdo);  
    LiberarArbolAVL(raiz^.Derecho);  
  
    // Liberar correo asociado  
    if raiz^.Correo  $\neq$  nil then  
        Dispose(raiz^.Correo);  
  
    // Liberar nodo  
    Dispose(raiz);  
    raiz := nil;  
end;  
  
procedure TEDDMailSystem.LiberarArbolB(var raiz: PNodeB);  
var  
    i: Integer;  
begin  
    if raiz = nil then Exit;  
  
    // Liberar hijos primero  
    if not raiz^.EsHoja then  
        begin  
            for i := 0 to raiz^.NumClaves do  
                LiberarArbolB(raiz^.Hijos[i]);  
            end;  
  
            // Liberar correos del nodo  
            for i := 0 to raiz^.NumClaves - 1 do  
                begin  
                    if raiz^.Correos[i]  $\neq$  nil then  
                        Dispose(raiz^.Correos[i]);  
                    end;  
  
                // Liberar nodo  
                Dispose(raiz);  
                raiz := nil;  
  
            end;
```

21.2. Optimización de Recorridos

Listing 46: Recorrido iterativo (alternativa)

```
procedure TEDDMailSystem.RecorridoIterativoAVL (raiz : PNodeAVL;  
  lista : TStringList);  
var  
  Pila : array [0..999] of PNodeAVL;  
  Tope : Integer;  
  Actual : PNodeAVL;  
begin  
  if raiz = nil then Exit;  
  
  Tope := -1;  
  Actual := raiz;  
  
  while (Tope >= 0) or (Actual <> nil) do  
  begin  
    // Ir al nodo m s izquierdo  
    while Actual <> nil do  
    begin  
      Inc(Tope);  
      Pila[Tope] := Actual;  
      Actual := Actual^.Izquierdo;  
    end;  
  
    // Procesar nodo actual  
    Actual := Pila[Tope];  
    Dec(Tope);  
  
    lista.AddObject(  
      Format(' [ID: -%d] -%s ', [Actual^.Correo^.Id, Actual^.Correo^.Asunto]),  
      TObject(PtrInt(Actual^.Correo^.Id)));  
  
    // Ir al sub rbol derecho  
    Actual := Actual^.Derecho;  
  end;  
end;
```


22. Troubleshooting Fase 2

22.1. Problemas Comunes del Árbol AVL

Problema	Solución
Árbol desbalanceado	Verificar rotaciones después de inserción
Stack overflow en recorrido	Usar recorrido iterativo para árboles grandes
Factor de balance incorrecto	Actualizar alturas después de cada operación
Memory leak	Liberar nodos en PostOrden

Cuadro 5: Problemas comunes - Árbol AVL

22.2. Problemas Comunes del Árbol B

Problema	Solución
Nodo overflow	Implementar división correcta de nodos
Claves desordenadas	Ordenar después de inserción
Pérdida de hijos	Copiar punteros en división
Búsqueda incorrecta	Verificar índices de arreglo

Cuadro 6: Problemas comunes - Árbol B

23. Conclusiones del Proyecto Completo

23.1. Logros de las Tres Fases

- **Fase 1:** Base sólida con 7 estructuras lineales y dinámicas
- **Fase 2:** 3 estructuras jerárquicas con balanceo y ordenamiento
- **Fase 3:** Seguridad, verificación, compresión y análisis de redes

23.2. Resumen Técnico por Fase

Aspecto	Fase 1	Fase 2	Fase 3
Estructuras	7 lineales	3 jerárquicas	5 avanzadas
Funcionalidades	Básicas	Búsqueda $O(\log n)$	Seguridad
Reportes	6 básicos	3 jerárquicos	3 especializados
Líneas de código	3,500	1,700	2,100
Complejidad	$O(n)$	$O(\log n)$	Mixta

Cuadro 7: Comparación entre fases

23.3. Estructuras Implementadas - Total

1. **Fase 1:** Lista Simple, Lista Doble, Lista Circular, Pila, Cola, Matriz Dispersa (6 estructuras)
2. **Fase 2:** Árbol AVL, Árbol BST, Árbol B (3 estructuras)
3. **Fase 3:** Blockchain, Merkle Tree, Grafo, LZW, Logueo (5 estructuras)
4. **Total:** 14 estructuras de datos implementadas

23.4. Métricas Finales del Proyecto

Métrica	Valor
Líneas de código totales	7,300
Funciones implementadas	120+
Estructuras de datos	14
Reportes Graphviz	12
Casos de prueba	40+
Formatos JSON	4
Interfaces GTK	15+

Cuadro 8: Métricas finales del proyecto

23.5. Lecciones Aprendidas

- Importancia del balanceo automático en árboles (AVL vs BST)
- Trade-offs entre complejidad y rendimiento (Árbol B vs Merkle)
- Seguridad mediante estructuras inmutables (Blockchain)
- Verificación de integridad con hashing (Merkle Tree)
- Optimización de almacenamiento (Compresión LZW)
- Análisis de redes sociales (Teoría de Grafos)

24. Referencias

1. Cormen, T. H., et al. (2009). *Introduction to Algorithms*. MIT Press.
2. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++*.
3. Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*.
4. Free Pascal Documentation: <https://www.freepascal.org/docs.html>
5. Graphviz Documentation: <https://graphviz.org/documentation/>

6. AVL Tree Visualization: <https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>
7. B-Tree Visualization: <https://www.cs.usfca.edu/~galles/visualization/BTree.html>

25. Anexos

25.1. Comandos de Compilación

Listing 47: Script de compilación Fase 2

```
#!/bin/bash
# build_fase2.sh

echo "====- Compilaci n -EDDMail- Fase-2 -===="

# Verificar dependencias
command -v fpc >/dev/null 2>&1 || {
    echo "Error: -Free- Pascal-no- instalado"; exit 1;
}

command -v dot >/dev/null 2>&1 || {
    echo "Advertencia: -Graphviz-no- instalado";
}

# Compilar proyecto
cd src/
fpc -dDEBUG -dFASE2 -Fu../lib -Fi../lib -FE../bin \
    -o../bin/EDDMail_Fase2 EDDMail.lpr

if [ $? -eq 0 ]; then
    echo "      -Compilaci n- exitosa"
    echo "Ejecutable: -bin/EDDMail_Fase2"

    # Crear directorios
    mkdir -p ../reportes/Root-Reportes
    mkdir -p ../datos

    echo "      -Sistema- listo- para- ejecutar"
else
    echo "      -Error-en- compilaci n"
    exit 1
fi
```

25.2. Información del Proyecto

- **Desarrollador:** José Alexander López López

- **Carné:** 202100305
- **Curso:** Estructuras de Datos
- **Sección:** C
- **Institución:** Universidad de San Carlos de Guatemala
- **Facultad:** Ingeniería
- **Escuela:** Ciencias y Sistemas
- **Fase:** 2 de 2
- **Fecha de entrega:** 03/10/2025
- **Repositorio:** https://github.com/JoseArt777/-EDD-1S2025_202100305
- **Versión del documento:** 1.0